

单位代码： 10293 密 级： _____

南京邮电大学

专业学位硕士学位论文



论文题目： 基于微服务架构的教材征订系统设计与实现

学 号 1314120108

姓 名 王倩莹

导 师 程春玲

专业学位类别 工程硕士

类 型 在 职

专业（领域） 软件工程

论文提交日期 2022.12

Design and Implementation of Textbook Subscription System Based on Microservice Architecture

Thesis Submitted to Nanjing University of Posts and
Telecommunications for the Degree of
Master of Engineering



By

WANG QIANYING

Supervisor: Prof. CHENG CHUNLING

December 2022

摘要

在互联网技术飞速发展的大背景下，高校教材征订系统在使用人数、专业种类上的需求呈现爆发式增长，高校对教材管理工作要求越来越精细化，传统教材征订系统耦合度高、扩展性差、高并发性能较差的问题随着爆发增长的用户量愈发明显。因此，本文针对传统教材征订系统高耦合、难以扩展、并发性差的特点，以优化系统应对高并发业务能力为目标，结合学校需求，通过微服务架构设计了一款工作效率更加高效的教材征订系统。本文的具体工作如下：

(1) 从国内外发展现状展开研究，分析了本文的研究背景与意义，介绍了系统构建时所用到的技术。

(2) 通过用例图详细分析系统需求将整个系统按照功能划分为六个微服务，其中有书架微服务、用户微服务、通知微服务、教材微服务、订单微服务、评价微服务。

(3) 使用 Eureka 完成服务的注册与发现，采用 Zuul 作为网关并整合 Ribbon 进行负载均衡、Hystrix 进行熔断降级，Config 作为配置中心，用 Spring Cloud 完成整个微服务项目的构建。最后本文对每个微服务用功能结构图加以说明，进一步使用类图、流程图和时序图描述每个微服务模块的详细设计过程。解决了传统单体架构可维护性差、扩展性差、部署不灵活等问题并优化了高校的教材征订系统处理高并发业务的性能。

(4) 系统进行功能性测试、服务组件测试和性能测试。最后从测试结果来看，本文设计的教材征订系统达到了预期功能，并且具有十分良好的高并发性能，同时也体现了微服务架构在教材征订系统中的优势与可行性。

关键词：教材征订系统，微服务，Spring Cloud，高并发，高校后勤管理

Abstract

Under the background of the rapid development of Internet technology, the demand for the college textbook subscription system in terms of the number of users and the types of majors is growing explosively. Colleges and universities have increasingly refined the requirements for the management of textbooks. The problems of the traditional textbook subscription system, such as high coupling, poor scalability, and poor concurrency, have become increasingly apparent with the explosive growth of the number of users. Therefore, in view of the characteristics of traditional textbook subscription system, such as high coupling, difficult expansion and poor concurrency, thesis aims to optimize the system's ability to cope with high concurrency business, and combines the needs of the school to design a more efficient textbook subscription system through the microservice architecture. The specific work of thesis is as follows:

(1) Based on the research status at home and abroad, thesis analyzes the research background and significance of thesis, and introduces the technology used in system construction.

(2) The whole system is divided into six microservices according to functions through detailed analysis of system requirements through use case diagrams, including user microservices, textbook microservices, bookshelf microservices, order microservices, evaluation microservices, and notification microservices.

(3) Thesis uses Eureka to complete the registration and discovery of services, and uses Zuul as the gateway and integrate ribbon for load balancing. Meanwhile, Hystrix is used for fuse downgrade, Config is used as the configuration center, and Spring Cloud is used to complete the construction of the entire microservice project. At last, thesis explains each microservice with functional structure diagram, and further describes the detailed design process of each microservice module with class diagram, flow diagram and sequence diagram. It solves the problems of poor maintainability, poor scalability, and inflexible deployment of the traditional monomer architecture, and optimizes the performance of the college textbook subscription system in dealing with high concurrency business.

(4) The system performs functional test, service component test and performance test. Finally, from the test results, the textbook subscription system designed in thesis achieves the expected functions, and has a very good high concurrency performance. At the same time, it also reflects the advantages and feasibility of microservice architecture in the textbook subscription system.

Key words: Textbook Subscription System, Microservice Architecture, Spring Cloud, High Concurrency, College Logistics Management

目 录

第一章 绪论.....	1
1.1 课题背景与课题意义.....	1
1.1.1 课题背景.....	1
1.1.2 课题意义.....	2
1.2 国内外研究现状.....	3
1.2.1 教材征订系统研究现状.....	3
1.2.2 微服务研究现状.....	5
1.3 本文主要工作和结构.....	6
1.4 本章小结.....	7
第二章 相关技术知识	8
2.1 微服务架构.....	8
2.2 微服务架构关键组件.....	10
2.2.1 服务注册与发现.....	11
2.2.2 API 网关.....	12
2.2.3 服务配置中心.....	13
2.3 远程过程调用.....	14
2.4 负载均衡技术.....	14
2.5 数据存储技术.....	15
2.5.1 MySQL 数据库.....	15
2.5.2 Redis 缓存.....	16
2.6 本章小结.....	16
第三章 教材征订系统需求分析	17
3.1 系统分析与目标.....	17
3.2 系统角色和功能分析.....	18
3.2.1 师生用户用例.....	19
3.2.2 教材科用例.....	20
3.2.3 教务处用例.....	22
3.2.4 系部用例.....	23
3.2.5 出版社用例.....	24
3.3 系统非功能需求分析.....	24
3.4 本章小结.....	25
第四章 教材征订系统设计与实现	26
4.1 基于微服务的教材征订系统总体设计	26
4.2 服务组件设计与实现.....	28

4.2.1 服务注册与发现设计与实现.....	28
4.2.2 服务网关设计与实现.....	30
4.2.3 配置中心设计与实现.....	31
4.3 基于微服务的教材征订系统高并发性能分析	32
4.4 教材征订系统服务功能设计	34
4.4.1 用户服务功能设计	35
4.4.2 教材服务功能设计	35
4.4.3 书架服务功能设计	36
4.4.4 订单服务功能设计	37
4.4.5 评价服务功能设计	37
4.4.6 通知服务功能设计	38
4.5 教材征订系统业务服务的详细设计与实现	39
4.5.1 用户服务详细设计与实现.....	40
4.5.2 教材服务详细设计与实现.....	44
4.5.3 书架服务详细设计与实现.....	47
4.5.4 订单服务详细设计与实现.....	49
4.5.5 评价服务详细设计与实现.....	51
4.5.6 通知服务详细设计与实现.....	52
4.6 数据库设计.....	53
4.7 本章小结.....	58
第五章 教材征订系统的测试与效果展示	59
5.1 系统核心功能测试.....	59
5.1.1 用户模块测试.....	59
5.1.2 教材查询订购模块测试.....	61
5.1.3 教材评价模块测试.....	63
5.2 服务组件测试.....	64
5.2.1 服务注册中心测试.....	64
5.2.2 网关服务测试.....	65
5.2.3 配置中心服务测试.....	65
5.3 系统性能测试与结果.....	66
5.3.1 服务端测试.....	66
5.3.2 客户端测试.....	67
5.4 本章小结.....	69
第六章 总结与展望	70
6.1 总结.....	70
6.2 展望.....	71
参考文献.....	72

专用术语注释表

缩略词说明:

ESB	Enterprise Service Bus	企业服务总线
XML	Extensible Markup Language	可扩展标记语言
B/S	Browser/Server	浏览器/服务器模式
JSP	Java Server Pages	JAVA 服务器页面
MVC	Model View Controller	视图，模型，控制器
API	Application Program Interface	应用程序接口
RPC	Remote Procedure Call	远程过程调用
MSA	Microservice Architecture	微服务架构
SOA	Service-Oriented Architecture	面向服务的架构

第一章 绪论

当前，互联网技术日新月异，高校教育蓬勃发展，传统架构的教材征订系统由于系统不易更新维护，扩展性较差已经不能满足系统新增业务和高吞吐量下的性能需求。本章介绍本文的研究背景与研究意义，分析国内外研究现状。结合高校教材征订的实际情况，明确了本文的主要研究内容以及章节的结构安排。

1.1 课题背景与课题意义

1.1.1 课题背景

目前，“互联网+”高校信息化建设已为高校师生带来了诸多便利，同时也使得高校工作效率得到了极大的提高。由于国内高校普遍采用学分制，大量选修课也另需开设，教学管理的复杂度也随着教学工作量的增长而不断提升。因此，教材征订在高校教务管理事务中举足轻重^[1]，其角色功能涉及教务管理人员、任课教师、出版社、学生等^[2]。同时随着政策的变动、专业的快速更替，不同的专业教材几乎不同，以致学校对征订管理教材有较大需求。

为了方便管理数量庞大的教材，高校教材征订一般使用诸多工具例如微软的 Office、金山的 WPS、Apache 的 OpenOffice、Oracle 的 Libre-Office 等。这项工作十分繁琐，需要由上至下多次通知和由下至上多次信息采集汇总，因此采用人工操作在大量繁琐的任务量前极易出错，其错误也难以排查。如今，大部分高校都采用了信息技术^[3]，通过教材征订系统来提高管理工作的效率。

现有的高校教材征订系统大多是基于单体架构进行开发。其中，以 MVC 架构应用最为典型。这种系统架构面对教材征订系统使用人数不断增长，应用程序规模随之不断拓展，基于单体架构的教材征订系统面临着内部耦合度高，水平扩展性差等问题^[4]。同时，基于单体架构的教材征订系统缺乏处理庞大请求量的能力，当系统运行在高并发环境时容易出现死锁，响应速度慢等问题。特别是系统中某一模块运行出现异常，则系统中其他功能模块也将受到影响^[5]。若是需要增加功能模块，实施也会较为复杂。大型企业通过高成本的硬件服务部署来解决高并发下系统易崩溃的难题，但这种解决方案成本高昂，不适用于高校信息化建设场景。本文在现有的高校教材征订系统基础上，需要增加例如教材评价的功能模块，而全校 3000 名师生需要在学期结束规定的短时间内完成本学期的教材评价，使得系统会在同一时间收到

大量的评价请求。因此，本文通过软件的方式解决教材征订场景中的高并发问题。

负载均衡技术通过平衡大量访问请求，分摊到多个服务器组件上执行工作任务，从而解决并发量高导致系统单点故障问题，是提高网络应用处理能力的关键技术。应用在单体架构中的负载均衡技术虽然能有效缓解系统的压力，但是也造成了极大的资源浪费。微服务架构^[6]通过搭建微服务应用相关组件运行在多个服务器中，或者独立运行同一种服务模块，并发处理海量请求数据，显著提高了负载均衡效率。此外，微服务架构将单个应用程序分解为围绕业务功能建立的一组可独立开发和部署的小型服务，有效解决了单体架构难以动态地满足用户不同需求的问题。由各微服务应用联合实现各功能，还具有独立部署、敏捷开发等优势^[7]。

本文通过调研挖掘单体架构和微服务架构各自的优势与劣势。现将两种开发模式结合高校教材征订系统业务进行对比分析，见表 1.1。

表 1.1 面向教材征订系统的单体架构和微服务架构的对比分析

对比维度	单体架构	微服务架构
业务耦合度	用户服务、教材服务和订单服务等各类业务耦合度高	用户服务、教材服务和订单服务等各类业务细粒度高、耦合度低
水平扩展性	对于相似业务场景，如图书订阅系统，扩展性差，可复用性低	适用于图书订阅系统、选课系统等相似业务场景，可复用性高
服务器性能	独立部署服务器，容易导致无法快速响应用户需求的问题	部署多台服务器分担计算任务压力
部署成本	随着教材征订系统的应用与发展，测试成本、构建成本和维护成本不断提高	相对于单体架构，教材征订应用开发、部署、运维成本较低

综上所述，本文首次将微服务架构应用在教材征订场景中，提高了教材征订系统在业务耦合度、水平扩展性、服务器性能、部署成本等方面的表现，应对短时间内处理大量用户并发访问的情况，不仅有利于提升高校教材管理质量与效率，还有利于为高校信息化建设夯实基础。

1.1.2 课题意义

教材是高校教学工作中的重要内容，也是高校传授知识的关键工具。处理好教材征订工作能够有力的促进学校教育改革，显著的提高教学质量和水平。

现实中高校教材征订系统大都采用单体架构, 将一个软件系统的功能模块跟运行的数据等全部看作一个整体, 统一地设计、开发、打包跟部署运行^[8], 没有针对功能进行模块化的细分, 代码庞大且耦合度高^[9], 灵活性低、扩展性差。微服务基本思想是把之前的传统单体应用根据功能拆分成若干能够被单独设计、开发、部署、运维的服务单元, 服务之间协同配合来实现最终价值^[10]。微服务架构轻松地解决了传统单体架构存在的缺点, 并且很大程度上提高了可拓展性和灵活性, 大大提升了系统的性能, 增强用户的体验感, 同时也提升了系统应对高并发业务的能力。面对教材征订系统数据较多, 功能日趋复杂的特点, 将微服务架构技术应用在其中, 对提升教材管理信息化建设水平具有一定价值。

本文的选题面向高校的基于微服务架构的教材征订系统设计与实现完成有着重要意义。

一方面, 从教材征订本身来说, 实现教材征订系统中的教材查阅、教材购买、教材测评以及工作流程的自定义, 使不同类型的教材征订工作能被教材征订系统很好地辅助, 有效地解决了教材征订系统代码实现跟随征订流程变更或所需功能模块的变更而改变的问题, 使其对校园办公起到重要的辅助作用, 极大地提高工作效率。

另一方面, 传统的单体架构应用在面临当前互联网环境的快速迭代和用户需求的迅猛变化情形时逐渐式微^[11], 新时代信息化发展和建设提出了高内聚低耦合的系统结构理念, 现有单体系统浪费分布式资源、可维护性低、后期可扩展性差等缺点与时代愈发脱节, 提高了项目的生产运营成本^[12]。因此, 传统单体架构逐步由更加灵活和具有更好可扩展性的微服务架构代替。目前, 微服务架构的优势越来越显著, 尤其在解决大型软件的复杂性问题, 微服务架构成为了更多企业设计大型系统的首选, 例如 Amazon 的网站、腾讯的微信系统、Netflix 的在线服务系统等^[13]。这些企业在项目中使用微服务的成功证明了微服务具有巨大的社会价值和市场价值。如上述, 微服务架构与高校教材征订系统的结合将具有一定的现实意义和社会价值, 因此本项目的实验开发也由传统单体高校教材征订系统转向微服务架构。

1.2 国内外研究现状

1.2.1 教材征订系统研究现状

高校信息化建设受到许多发达国家的格外重视, 这些国家拥有技术相对稳定的庞大规模团队提供服务跟技术的支持, 通过高带宽的特定网络, 制定教育目标, 修改教育改革方案, 建设教育信息化。许多学校对现代信息化教材管理系统进行深入的研究与开发, 现在科学系统管理模式已经日臻成熟。在美国, 有 86% 的高校能够在线提供课程目录, 有 75% 的高校具

备学生在线注册课程的服务^[14]；欧洲一些国家，现代化的教学平台已经出现在大多数学校^[15]。西方发达国家的高校教材管理系统满足了当下大部分高校对教材管理的需求，他们的教材管理系统十分可靠并且相当稳定。

Abraham Seidmann 借助 PHP 以及相关数据技术，结合多种线上移动支付方式，设计出了一套能够开展教材基础信息管理、征订以及订单管理的信息化管理系统，大大提升了高校教材信息管理的便捷性。国内较早使用 JAVA 语言对高校教材管理系统后端进行设计与实现；基于 B/S 架构使用 JAVA 语言从系统设计和实现方面对高校教材征订系统的实现进行了叙述^[47]；同时使用 C#语言并采用 ASP.NET 框架实现了 B/S 架构的教材征订系统^[47]。Fuminori Toyasaki 为有效规避高校教材管理系统的耦合度，基于 MVC 架构，开发出一种高校教材管理系统，提升了高校教材管理系统的出入库管理能力以及订购能力。以上教材管理系统进行设计时都基于传统架构或 MVC 架构设计理念，但系统在大规模业务、高并发访问情况下不能很好的处理，因此本文采用分布式思想解决教材管理系统中存在的高并发业务处理不及时的问题。

由于国内与国外在发展环境以及文化生活上存在着一定的差异，导致国内外教育方式与理念存在一定的隔阂，这些差异同时也体现在教学管理方面。所以，我国相关研究者应该以国内高校的实际状况为基础，借鉴西方发达国家的科学经验，制定出符合国内教育环境的教材征订系统。目前，教材征订系统形式一般只有下面几种：一是相对独立的子系统，该系统结构简单，作用单一，仅仅能够满足购买教材。二是比较完善的系统，这种系统是由专业的开发团队进行研发，其优点有流程独特、灵活性高；同时其缺点也一直为人诟病，比较死板。第三种系统采用的比较频繁，通过校企合作开发，根据自身需求研发的一套教材征订系统。这种系统基本满足了高校的教材管理的需求，但是为了缩短工期，系统开发时大都采用统一的模板进行开发，导致了系统的界面大同小异，难以满足个性化定制且比较繁杂，并且由于系统各组件过于紧密的耦合性，造成后期维护不便。

目前看来，各级教育机构的教材管理系统还处于一个与西方发达国家有一定差距的初级发展阶段。由于每所高校教材管理部门都存在一定的差异，征订流程也各不相同，系统所需要的功能也有很大区别，因此以上介绍的每一种教材征订系统都不能适用于所有高校。所以高校教材征订系统在参考西方发达国家的发展经验与教训的同时也需要结合其自身的实际需求，确立明确的研发方向，完善实验，进而达到预期的目标，实现高校教材征订的需求。

1.2.2 微服务研究现状

最初,传统单体架构系统为了满足能够快速实现业务功能的需求,着力于构建单一、集中式的全功能开发。但由于业务变化、需求增加等一系列因素的影响,规模越来越大和更新迅速的新时代网络环境下的软件构建需求已无法通过单体架构系统满足。所以,众多软件开发者的重点放在了如何研发出灵活、易扩展并且能及时满足需求变化的系统架构^[18]。一套微服务管理架构即面向服务的分布式架构 SOA^[19]就可以很好的将多个核心业务抽取出来。由于灵活、高效、更适用于新时代互联网的优势,微服务架构渐渐走进大众的视野成为新的架构趋势^[20]。

Gartner 于 1996 年提出了面向服务的架构,这是微服务的前身^[21]。SOA 架构提出了可重用粒度的划分标准,将相互联系的功能进行组合并提供功能服务给消费者^[20]。2000 年,网络服务、企业服务总线、SOAP 等技术的出现,让 SOA 架构技术体系愈发成熟,越来越多的企业的解决方案以及产品都以 SOA 为基础。与此同时,伴随着容器技术^[18]、敏捷方法、DevOps^[23]、持续集成^[22]、云技术^[24]等方法论的提出,开发团队越来越注重软件系统响应业务需求变化的能力。使得开发组能够构建更快速、更可靠、能多频次发布的交付机制等^[25]。同时在各种因素的催化下,更加灵活和轻量级的 SOA2.0,即微服务架构 MSA 应运而生^[20]。

当今信息科学领域对微服务进行重点研究,工程界和学术界对其有着广泛关注^[26]。微服务这个概念开始是由 Martin Fowler^[27]等众多计算机专家的推动,是一种新兴的软件架构的设计风格与组织模式^[28]。微服务架构继承并发展自 SOA 架构,将系统分为众多更加轻量的微服务一起协同配合响应并完成用户提出的需求与任务。微服务架构与最初的 SOA 相比更加灵活且更易扩展,更加注重每个微服务能够独立部署、独立测试、独立运行的能力,也与 DevOps 更加契合,从而更加容易构建并投入企业进行应用。微服务的出现与软件系统微服务化的实践引出了多种研究热点,微服务划分、设计、测试、管理、部署等都是当今时代信息科学领域的重点关注对象^[29],同时 Docker 容器化技术的出现,最大程度上解决了微服务应用的环境创建难、配置难、部署难以及交付难等问题^[30],为微服务的商业化提供了保障。

目前 Spring Cloud、Zookeeper、Dubbo 等微服务框架的诞生和推广,实现了微服务架构快速构建企业级应用的目标,推动了微服务的广泛应用^[21]。根据 2018 年全球微服务趋势报告^[13],预计使用微服务的企业约占 91%,采用微服务的企业大概占 60%,目前使用微服务技术生产微服务架构模式的应用程序的企业大概有 29%。不仅如此,在 2017 年有着下一代微服务称号的服务网格(Service Mesh)也顺应潮流诞生^[13],服务网格的强调重点在于服务之间的通

信以及能够和 DevOps 更加契合。服务网络的诞生是软件架构的探索历程将进入又一新的阶段的重要标志。

1.3 本文主要工作和结构

本文的教材征订系统是基于微服务架构开发的，通过微服务架构的长处来优化单体架构应用的一些不足，将教材征订系统各个功能业务模块充分解耦，使得系统更加可靠而且更加容易扩展。

为了使得教材征订系统微服务化，本文主要通过以下四点进行研究：

- (1) 研究了微服务架构的优势，设计了以微服务架构为基础的教材征订系统的设计方案；研究了教材征订系统实现所需的各项技术。
- (2) 实现了教材征订系统基于角色的功能划分与详细设计，针对所需要的功能进行系统的功能分析并根据业务逻辑划分了功能模块，采用微服务架构设计了组件化的业务功能。
- (3) 完成详细设计后，通过微服务架构技术、数据库技术、java 开发技术实现所需要的各项业务功能，其中研究面向高并发的微服务负载均衡技术。
- (4) 测试教材征订系统的主要功能，分析测试结果验证了基于微服务的教材征订系统的可用性与有效性。

本文通过六个章节来论述，各章节按如下内容安排：

第一章，绪论部分，主要介绍教材征订系统的背景和意义，微服务架构发展历程以及实际运用状况，以及本文的主要研究方向。

第二章，系统相关技术知识的介绍，阐述了微服务架构技术，概述了教材征订系统中采用的各项技术，并对相关技术的原理进行了说明。

第三章，教材征订系统需求分析部分，阐述了需求背景和要求，叙述了系统中的角色划分。从用户角色的需求出发，建立了五个用例规约，通过用例图对其进行功能分析，确立教材征订系统所需要实现的功能。

第四章，教材征订系统详细设计与实现部分，按照需求分析对教材管理系统的微服务总体架构模型进行了设计。划分六个微服务来满足教材征订系统的业务功能，并对每个微服务用功能结构图加以说明。接着使用类图、流程图、时序图详细叙述了各个功能服务的设计与实现，并进行了相应的数据库设计同时使用了面向高并发的微服务负载均衡技术。

第五章，对本文设计的教材征订系统进行测试与实验，通过测试用例测试了教材征订系统的功能，并根据系统的测试结果进行性能分析。

第六章，总结与展望，对全文的内容进行总结，分析并找出本文所设计的系统的不足之处，并对未来的优化和研究方向作出展望。

1.4 本章小结

本章是全文的绪论，首先介绍了当前高校教材征订管理现状、教材征订管理对高校日常工作的重要性、传统教材征订系统的不足以及优化这些不足之处的方法，然后介绍了国内外微服务发展以及现实应用。最后对本文的研究内容进行了介绍，介绍了本文的组织结构并确定了本文的主要工作方向。

第二章 相关技术知识

在开发信息管理系统时,结合征订系统所需的与用户进行交互、对教材征订信息进行管理以及便于后期维护扩展的目的选择了微服务架构来开发系统,进而确定使用 Spring Cloud。本章介绍了基于微服务架构的教材征订系统所用的开发技术以及原理。包括负载均衡技术、数据库关键技术、微服务架构简介、微服务架构组件和 RPC 远程调用。

2.1 微服务架构

随着软件技术的迭代,目前系统架构为表示层、业务逻辑层与数据访问层所代表的三层模型^[31]。在传统的单体架构中一个工程项目中通过编译,打包部署将三层模型打包到单一服务器中。很多公司为控制后期运营成本在软件开发的初级阶段采用简单易搭建的单体架构。然而,单体架构随着业务的不断发展,需求的不断增多,项目愈发复杂其缺点也逐渐显现^[32]。单体架构的缺点主要有:单体架构系统随着业务功能的增加、用户访问量的增多,系统可靠性会越来越差^[33];由于所有代码都在一个项目里,系统服务端业务代码跟和单体架构搭建的项目代码量越来越多,大大降低了代码的可读性与可维护性^[34]。

微服务的诞生有效解决了单体架构的一系列缺陷,微服务架构(MicroServices Architecture, MSA)指通过对预定义的微服务进行重新组建形成企业级应用来满足系统业务需求的分布式体系结构^[35]。微服务中心思想是“高内聚低耦合”,由 Martin Fowler 与 James Lewis 共同提出的^[36],原理是将原系统按照业务模块划分成微服务子系统,提高系统功能的可扩展性与可维护性^[37]。整个业务模块的最小粒度即为微服务,即微服务是单个业务模块的拷贝^[38]。架构演进经历了从单体架构到 SOA 架构再到微服务架构;随着微服务架构的广泛使用,微服务架构也逐步成为当前软件架构的潮流^[39]。微服务架构的主要特点有以下四个:

(1) 功能模块独立运行

微服务架构可以将复杂的系统根据业务模块拆分成相互独立的微服务^[40],根据业务功能划分为多个模块后,各个模块独立的运行,各微服务之间彼此解耦^[41]。增强了系统的稳定性的同时也防止了单个业务模块异常而造成系统死机,有助于开发人员快速定位问题所在并解决。服务还可以使用不同的语言进行开发^[42]。本文的教材征订系统功能多样且繁杂,采用微服务架构可以按照征订系统的需求将功能划分为多个微服务。这样可以使得每个服务相对较小并且容易维护,开发者可以更容易理解各服务中的逻辑代码,服务的代码量较小也为软件

测试提供了较好的基础。在系统运行时，小的代码库使得服务启动的速度也变快了。征订系统采用了微服务架构后，系统的各个服务可以独立进行部署与扩展，如果想对征订系统的功能进行修改或者添加新的功能，只需要修改与其功能相对的功能模块即可，不需要牵动整个系统，这使得征订系统后期的更新与维护变得及其方便。得益于此特点，本文的征订系统能够快速完成了相关的系统设计且独立开发出来。

（2）通信方式

微服务架构的通信方式一般使用 HTTP。HTTP 协议的一般流程为：发送方发送请求，接收方对收到的请求进行处理，返回数据。其中通信时数据采用 JSON 格式传输，简化了数据访问流程^[43]。

（3）相互独立的数据库

传统的单体架构的业务都依托一个数据库来完成。单一数据库中的表会随着业务的增多而增加，表的增多会使数据的查询、修改变得缓慢。而微服务架构相较于单体架构的优势是一个服务专享一个数据库，方便了管理和维护^[44]。相互独立的数据库对于教材征订系统后期维护的便利同样起着不可或缺的作用，当开发者要对某一功能进行优化只需找到对应的数据库对其进行维护即可，不会影响其他的数据库也不会使得整个系统的数据变得越来越繁杂，相当于使其功能变得强大但不会对它的使用情况进行打折。本文的征订系统也运行在多个数据库上，征订系统本身的数据库及其接口可以较为轻易的被其他系统调用。

（4）服务治理

单体架构并没有服务治理这个概念，所有功能全部集中在一个模块中。而微服务相较于单体架构，根据各业务划分出各个服务，管理随着服务增多变得越发复杂，所以有了服务集中治理^[45]。本文的征订系统功能多样繁杂，后期随着需求增加而维护频繁，微服务架构的服务治理，恰恰缓解了征订系统所面临的这一问题，服务治理这一功能还非常有利于大型复杂系统的开发。

本文的征订系统使用微服务架构除了因为上述四个特点之外，在开发一些远比征订系统更加复杂的软件时具有这样的优势，即微服务架构可以有效实现团队的自治，在开发复杂功能软件时按照不同的功能划分微服务给不同的团队，各团队负责属于自己的任务，这使得系统的开发平稳稳步进行，缓解了整体的压力，同时使得各个板块都得到重视。不仅如此，微服务架构可以消除对某一个技术栈的长期依赖。因划分为不同的微服务，服务变得更小，使用新的编程语言以及新的技术对某一项任务进行重写成为可能。这样的话，当开发者开发复杂系统软件的时候，当某一服务模块的一项技术尝试失败后，可以直接丢弃这部分而且不会

给整体带来失败的风险。微服务架构作为分布式架构的代表之一，错误会由于网络问题以及其他原因而发生^[46]。微服务采用弹性设计，对系统进行了容错处理。综上所述，微服务架构十分适合征订系统和其他大型复杂系统的开发。

微服务架构已经在好多企业逐渐流行起来了，成为了当下的一种趋势。如 Spring 团队的 Spring Cloud、腾讯 Tars、阿里巴巴的 Dubbo 等^[47]。本文的服务端架构是通过 Spring Cloud 来搭建与维护系统的，Spring Cloud 基于 Spring Boot 提供了一整套服务的解决方案，包括服务注册与发现，配置中心，全链路监控，服务网关，负载均衡，熔断器等组件。Spring Boot 是在 Spring 4.0 的基础上生成的^[48]，它的作用是使开发者开发的更加快速，更加敏捷从而快速的搭建 Spring 框架。Spring 框架具有 IOC（控制反转）和 Aop（面向切面编程）两大特点，Spring 的 IOC 容器，能使 Spring 进行对象控制，降低程序代码的依赖性^[49]，同时让代码之间的耦合性降低。Aop 能起到面向切面编程的作用，使系统的复杂度降低，使开发者有一个更加清晰的业务逻辑。由于 Spring 框架配置繁琐，目前大多项目采用 SpringBoot 替代：Spring Boot 通过使用 Java Config 的方式简化了配置；Spring Boot 通过提供大量的基础组件，使得现在框架技术更加方便使用；Spring Boot 创建 Web 文件自动的添加依赖不同于 Spring 需手动添加多个依赖；Spring Boot 由于内嵌 web 服务器，可以开箱即用直接运行，不需要使用另外的容器如 Tomcat^[50]。

2.2 微服务架构关键组件

Spring Cloud 为微服务架构中的配置管理、智能路由、服务治理、断路器、微代理、分布式会话和集群状态管理等操作提供了一种简单且便捷的开发架构^[51]，极大地简化了分布式系统基础设施的构建工作^[52]。

Spring Cloud 的一套功能是由许多组件来支撑的^[53]，包括：服务注册中心 Eureka，它是用于服务注册与发现机制的服务治理组件；服务调用组件 Ribbon，它实现了服务端的负载均衡；服务调用组件 Feign，它是以 Ribbon 和 Hystrix 的声明式为基础运行的；智能路由、安全认证等功能的网关组件^[54]Zuul。Spring Cloud 组件架构图如下图 2.1 所示。

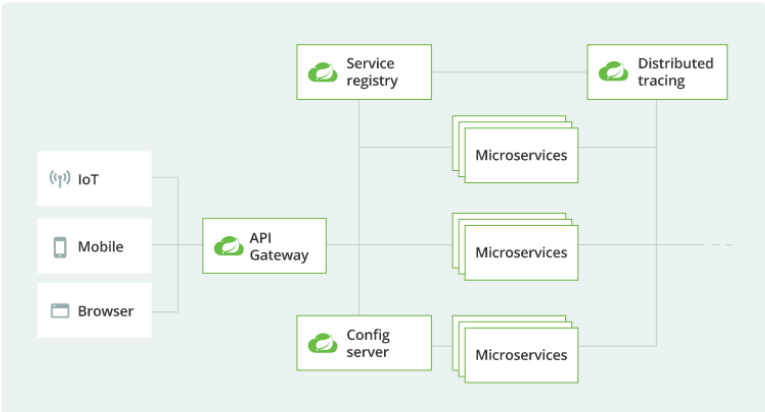


图 2.1 微服务组件架构图

本系统主要使用 Spring Cloud 框架中的三个组件 Eureka、Zuul、Config，其中 Eureka 可与 Zuul 配合使用。

2.2.1 服务注册与发现

服务注册与发现的机制使用户通过注册就能够发现其他已经进行注册了的用户^[55]。Eureka 在 Spring Cloud 中实现服务发现与注册功能。Eureka 是以 REST 服务为基础的一个组件，是由 Eureka Client 和 Eureka Server 组成的。Eureka Client 是一个承担上报服务信息，拉取服务信息，自动解析等功能的 Java 客户端；Eureka Server 是缓存服务数据和提供服务数据的服务注册中心^[56]；Client 通过按时给 Server 报告心跳的方式来证明自己的存活^[57]。各个微服务在启动的时候，首先 Eureka Client 会向 Eureka Server 发起注册，Eureka Server 则会把此服务的信息存储起来。所以 Eureka Server 有每个微服务的客户端和服务端的注册信息，并且微服务的客户端与服务端之间可以相互注册^[35]。Eureka 组件原理图如下图 2.2 所示。

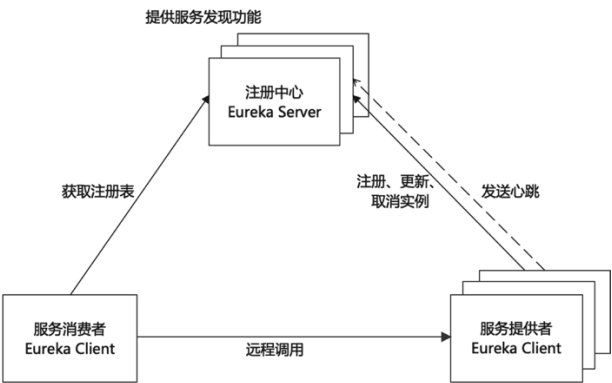


图 2.2 Eureka 原理图

Eureka 有三大优势。首先，Eureka Server 以互相注册的方法组建注册中心，通过服务端实例互相复制以同步注册中心数据；其次，Eureka Client 会对服务注册表中的信息进行缓存，

这使得微服务无需每次从注册中心获取提供者信息,从而减少了注册中心的交互频率;最后,假如因为网络原因 Eureka 丢失大量心跳连接,系统在这种情况下会进入自动保护模式。

2.2.2 API 网关

在单体架构中,API 网关是客户端与服务端采用直接连通的通信方式。客户端在发送一个请求后都会对服务器上相应的服务进行直接访问,访问效率会随着请求增多而降低。在微服务中,一个独立的系统被拆分成了很多个独立的服务,除了系统访问效率,为了确保安全,权限管理也是一个不可回避的问题,如果在每一个服务上都添加上相同的权限验证代码来确保系统不被非法访问,那么工作量也就太大了,而且维护也非常不方便。为了解决上述问题,微服务架构中提出了 API 网关的概念,它就像一个安检站一样,所有外部的请求都需要它来实现负载均衡、权限验证、请求路由等功能,然后到达后端微服务,可以使得客户端和各微服务之间的交互频率降低。每个服务处理完请求后,通过 API 网关将传送回的结果发送给客户端。API 网关 Zuul 架构图如下图 2.3 所示。

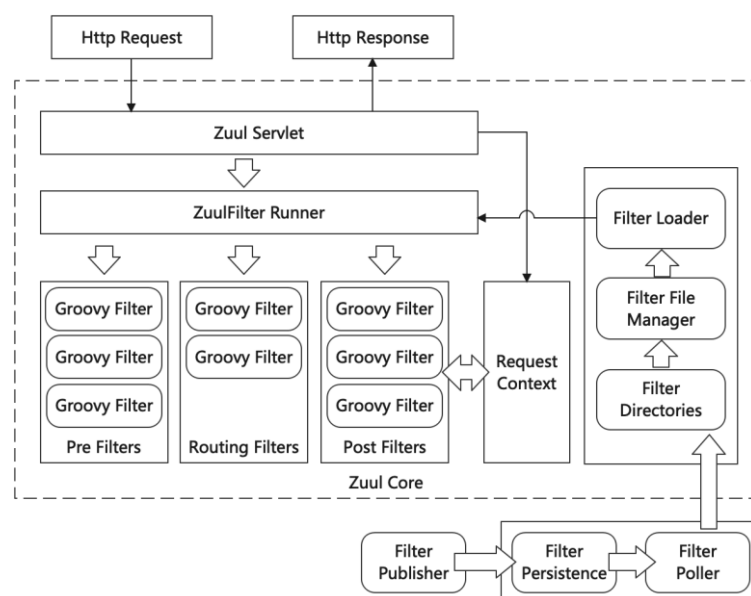


图 2.3 Zuul 架构图

API 网关是整个系统的唯一访问入口,外部访问都需要通过一个统一的 API 网关,然后再访问相应的微服务,这是 API 网关所具有的最明显的特点^[58]。本文选用 Zuul 作为服务网关,Zuul 对 HTTP 请求的处理核心就是 ZuulServlet 类,而 ZuulServlet 本质上是符合 Java EE 规范的 Servlet 实现类。Zuul 的核心处理逻辑由一系列 Filter 组成,而这些 Filter 全部封装在 ZuulServlet 类的 Runner 中,Runner 在接收到 HTTP 请求后会依次调用 ZuulFilter,而 ZuulFilter 则是 Zuul 在初始化时加载到 FilterProcessor 的 Filter 实例。

Zuul 最主要的工作机制是基于 ZuulFilter 的链式调用请求机制, ZuulFilter 之间没有直接的通信, 它们的数据传递只能依靠过 RequestContext 类。RequestContext 类中通过 ThreadLocal 变量来记录每个 Request 所需要传递的数据。ZuulFilter 可以使用 Java 或者 Groovy 动态代码实现, 同时使用 Filter Loader 的动态文件夹轮询扫描功能, 可以将 ZuulFilter 动态加载到 FilterProcessor 中, 实现动态 Filter 功能。

2.2.3 服务配置中心

在传统的中心化单体架构中, 所有的配置项都是通过本地的静态配置文件进行管理的, 对于不同的环境, 需要手动维护和切换调整不同的配置。在分布式微服务环境下, 一个系统往往由多个微服务组成, 每个微服务都需要独立的配置文件, 那么就需要同时管理许多配置文件, 低效且易出错。那么服务配置中心就是将各种配置参数全部集中到一个地方进行统一管理, 并提供一套标准的接口规范。当不同服务需要获取参数时, 可以从配置中心拉取并配置, 当修改配置时, 可以由配置中心统一下发给集群中的所有实例^[59]。Config 组件结构如下图所示 2.4 所示。

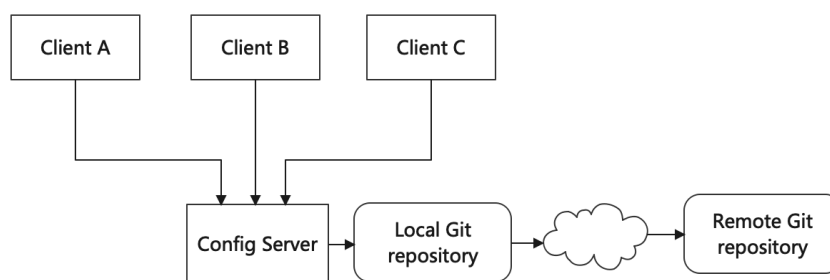


图 2.4 Config 架构图

Spring Cloud Config 提供了服务端和客户端的支持给分布式系统配置, 其中有 Config Server 和 Config Client 两个部分。Config Server 用于对应用程序各个环境下的配置进行集中管理, 是一个集中式的并且可以横向扩展的配置服务器, 存储配置内容默认使用 Git, 因此可以控制配置的版本以及实现对内容的审计。Config Client 提供了基于 Spring 的客户端, Config Server 负责把 Git 上的配置返回客户端。配置信息会存储在一个远程 Git 仓库中, Config Server 接到来自客户端的配置获取请求后, 先把远程仓库的配置克隆到本地的临时目录, 然后从临时目录读取配置并返回。Spring Cloud Config 通过配置服务 (Config Server) 来为所有的环境和应用提供外部配置的集中管理, 也能对应用的开发、测试、生产环境的配置做切换、迁移^[59]。

2.3 远程过程调用

远程过程调用 RPC 是一种用于进程之间的通信方式。通过该技术，不同机器上的程序可以在共享网络之间调用，舍去了开发人员额外编写程序调用的流程。这样不仅减少了工作量还实现了隐私保护。RPC 的原理如下图 2.5 所示。

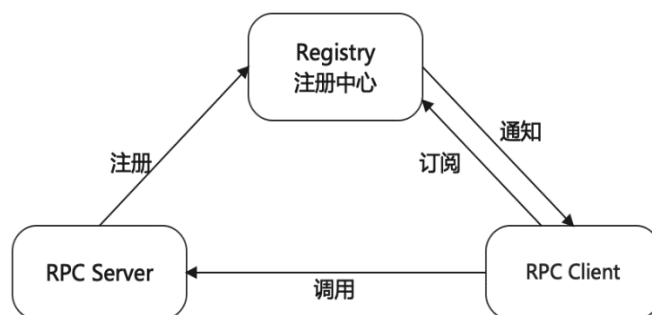


图 2.5 远程过程调用（RPC）原理图

RPC Server、RPC Client 与注册中心 Registry 是 RPC 的重要组成部分。其中 RPC Server 为用户提供服务，RPC Client 可以调用 RPC Server 上的服务，RPC Server 上的服务可以在 Registry 注册中心进行注册，RPC Client 有两种途径从 Registry 注册中心获取可用的服务。RPC Client 一般先向 Registry 获取提供的服务列表再调用服务，然后 RPC Server 启动并注册机器 IP、port 和相关的服务。如此一来 RPC 技术完成完成了远程调用。

2.4 负载均衡技术

负载均衡技术是解决高并发问题的一项重要技术，其遵循负载均衡策略并综合考虑各个节点服务器资源的性能，使各个节点更好地协调工作，在减少响应时间的同时提升用户体验。具体来说，面对来自客户端的大量用户访问请求，负载均衡根据负载均衡策略合理地将海量并发访问数据分发给服务端系统的节点服务器进行处理，以平衡服务端系统各服务器间的通信负载，避免服务器因为单点故障导致服务端系统崩溃的情况出现，从而增强服务端系统的数据处理能力，实现智能通信管理^[60]。

本文根据负载均衡实现的途径不同，将负载均衡技术的分类分为硬负载均衡技术和软负载均衡技术^[61]。

硬负载均衡技术主要通过负载均衡器来实现，如 F5 负载均衡器等。该技术也被叫做基于硬件实现的负载均衡技术。硬负载均衡技术因为有专门的负载均衡器来完成计算分担任务，所以具有自由度较高、独立性较强的特点。但因为硬负载均衡技术部署成本比较高昂，一般

只有大型企业才能成功应用^[62]。在实际应用中，负载均衡器可以被搭建在交换机、服务器与外部网络间或者两台服务器之间，具有多样化的负载均衡策略，能够智能化管理流量。

软负载均衡技术主要通过在服务端部署负载均衡软件应用来实现负载均衡。也被叫做基于软件实现的负载均衡技术。软负载均衡技术具有配置简单易行，成本相对低廉，使用方式灵活等优势。然而，软负载均衡因为自身资源的局限性，极易成为系统性能的瓶颈技术^[63]。其中，就 SpringCloud 中的 Ribbon 来说，在面对大量用户的访问请求时，不仅要处理这些请求，还需要分析反馈到客户端，所以软负载均衡技术会占用服务端的一定资源。越是高效的软负载均衡技术，要占用的服务端资源也就越多，不利于提高服务端系统的整体吞吐量。同时软负载均衡技术还通过多种算法来实现服务器之间的资源调度管理包括：轮转调度法、加权轮转调度法、随机均匀调度法、源地址散列调度法等。本系统采用加权轮转调度法作为 Ribbon 的负载均衡算法。

2.5 数据存储技术

在本系统的开发过程中，系统使用 MySQL 数据库来实现持久化存储，而需要快速响应的数据则使用高性能的 Redis 进行存储。Redis 缓存技术为系统的高并发应用环境提供了技术保障。

2.5.1 MySQL 数据库

MySQL 是一种关系型数据库通过关系模型来组织和管理数据。关系型数据库将各种类型的数据根据它们之间的关系有序地存储在不同的表中，因此具有查询方便、修改灵活等优点。关于如何选取本系统数据库，本文选用了一直被广泛使用、兼容性强的关系型数据库 MySQL。MySQL 可以在多个平台进行开发，可以在不同的平台进行移植，且不用对其进行改变。MySQL 使用了极快的 B 树磁盘表和索引压缩^[64]，能够以很快的速度进行连接，用高度优化类库实现 SQL 函数，有着很快的运行速度。

MySQL 适用于多类存储引擎，并且有多版本并发控制机制与事务。MySQL 能够满足不一样的运用场景，并且保障了数据库的安全性，因此运用了 MySQL 的结构化数据。MySQL 是一个体积小、运行效率高、能够运行在多种操作系统、可以使用不同的编程语言的关系型数据库管理系统。MySQL 数据库开源的特征方便了广大用户，也是中小型数据库的首要选择。

MySQL 把查询处理等系统任务与数据存储、提取分割开来，使用这类存储引擎架构在

选择数据存储方式时，能够按照特性、性能以及其他要求来选择，这是 My SQL 最与众不同的特性。

2.5.2 Redis 缓存

传统的关系型数据库对于大量的数据查询请求会进行的很吃力，所以高速缓存就成为必不可少的存在。Redis 采用多路复用 IO 技术^[65]，具有非常好的性能，吞吐量可达每秒 10 万次。由于本系统中存在大量的数据查询请求，所以选用 Redis 来满足系统的高速缓存的需求，保障了多用户的并发使用。

Redis 是基于内存的键值缓存和存储系统。Redis 在运行时，所有的数据均以 Key-Value 的数据结构在内存中进行存放，故而可以极快的实现对数据的各类操作。同时 Redis 拥有可靠的持久化功能。除了速度，还需要拥有数据的持久化，AOF 和 RDB 是 Redis 提供的两种数据持久化方案，能够持久的对数据进行保存，机器在出现故障或者重新启动后能够恢复，使得数据变得安全可靠。

使用 Redis 缓存，能够使系统的运行效率大大提高，当收到服务的请求时，首先查找位于内存中的 Redis 缓存，这样使其具有很快的查询速度。如果资源存在于缓存之中，服务能够马上返回查找的资源，如果不在缓存之中则需要再次从网络端发起请求^[66]。

2.6 本章小结

本章对平台设计和实现过程所需要的关键技术进行了介绍。首先对系统中使用的数据库关键技术、微服务架构组件进行了说明，并对教材征订系统解决高并发问题的负载均衡技术进行介绍同时介绍了 RPC 远程调用与 Redis 缓存相关技术。

第三章 教材征订系统需求分析

伴随着高校教材征订系统用户越来越多,精细化工作带来的业务种类增加,教材征订系统所需功能也越加复杂。基于微服务架构的教材征订系统对于目前教材征订面临的并发量有限、扩展难和维护不方便等一系列问题有着很好的缓解作用。本章先是对教材征订系统的实现目标进行了说明;然后对系统的总体需求进行了阐述,具体介绍了功能性需求与非功能性需求;阐述了对于不同角色的业务功能,并通过用例图的方式说明了这些角色的功能需要。

3.1 系统分析与目标

本文根据高校的教材管理需要对教材征订系统进行设计,考虑到招生规模逐渐扩大,教务的业务系统需求逐步增加,目前的单体架构教材征订系统表现出种种局限性。这些局限性主要体现如下:

(1) 耦合度高。目前放开教材系统的所有模块共用一个数据库,这种方式导致过高的系统耦合性,带给数据库极大的安全压力和存储压力。而微服务将业务模块拆分,每个微服务享有一个专有的数据库,大大降低了系统的耦合度,提升了系统灵活性。

(2) 应用较复杂。随着业务和需求的增长,代码量会不断扩张,传统的单体架构系统维护模块化结构愈发困难,大大影响了后续的可维护性;而基于微服务的应用,模块与模块分开,大大提升了代码的可维护性,从而降低了故障率。

(3) 容易发生故障级联。传统一体化架构,部分模块的异常会导致整个系统崩溃,而微服务的分布式部署则能很大程度避免。

(4) 容易陷入语言陷阱。传统一体化架构只能用单一语言开发,不能发挥语言的优势。而微服务架构能使用不同语言开发,每种语言服务一个模块,充分发挥了不同种语言的优势,比如在需要科学计算算法的模块可以嵌入 python 语言。

(5) 并发业务效果不理想。在大量业务请求发送到系统时,传统单体架构的征订系统只能逐一处理不同模块业务请求,造成大量的时间资源浪费。而基于微服务架构的征订系统,通过控制面的转发,特定模块处理特定请求,并行处理的方式大大降低了系统的响应时间,提升了用户使用体验。

针对上述分析,我们在原有系统的基础上,基于微服务建立一套的征订系统,以进一步优化系统使其能够对外高效并且增强服务的扩展性。下面的功能是新的教材征订系统要具备

的:

(1) 能够维护教材系统的基本信息。要具备用户信息、教材信息这些基本信息,能够很高效的查询到这些信息。

(2) 权限管理。在老系统的权限管理的基础上,更好的分配系部、教师、学生、管理人员、出版社的权限。

(3) 操作简洁。需要一个更加简洁的操作页面来应对繁杂的系统征订量。

(4) 流程规范。对各类用户查询、订购、领取教材管理的流程进行设置,进一步规范教材征订管理。

(5) 提升管理系统的运行效率以及其扩展性。由于教材征订系统需要对接教务管理系统等其它系统,这就需要本文的系统有很高的扩展性并且能够给予其他系统服务,还要保持高效的运行状态。

总体而言,整个系统要有简便的操作方式,简单整洁的界面,简便的维护过程,有稳定运行的数据库,快速的执行速度,很高的数据安全性和高并发能力,后期容易扩展。

3.2 系统角色和功能分析

就高校教材征订系统而言,系统中的角色可分为学生、教师、教材科、教务处、系部与出版社六类。尽管角色种类繁多,但具体分析使用对象为普通用户和管理人员。其中普通用户主要由学生、教师、系部、出版社组成,完成基本的教材浏览、查询和评价等。管理人员由教材科与教务处构成,负责后台用户管理、教材管理和订单管理等。故本文所设计的高校教材征订系统整体功能结构如图 3.1 所示。

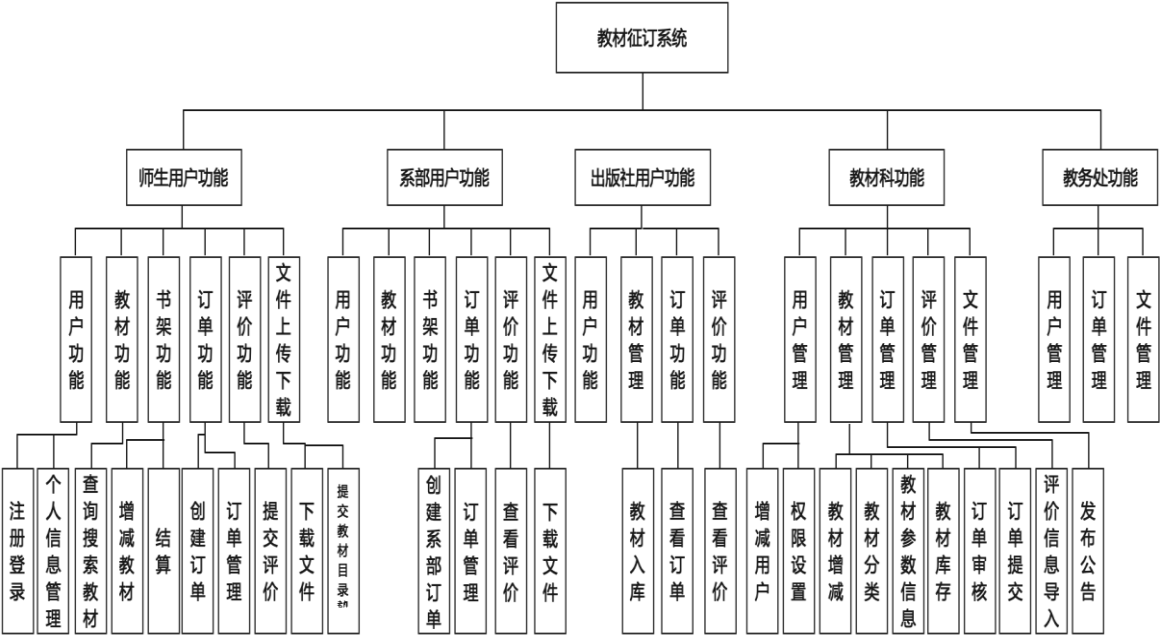


图 3.1 角色相应关系图

3.2.1 师生用户用例

教材征订系统客户端对普通用户开放，主要包括教师和学生用户。根据整理的普通用户功能需求，制作成表 3.1 用例规约，图 3.2 是相应的用例图模型。

表 3.1 师生用户用例规约

项目	内容
用例名	师生用户用例
参与者	教师用户、学生用户
描述	老师和学生用户能够注册账号，对账号的个人信息进行填写并完善；对个人账户信息进行管理；还能够及时对发布的通知（例如教学计划）进行浏览，按自身需求，查询、订购、领取、评价教材等。教师用户还可提交教材目录。
前置条件	师生用户注册并且成功登录，具有能够执行的工作流程定义。
触发	师生用户触发
后置条件	教材查询、订购、领取、评价成功
过程	1. 教师或学生用户注册账号，填写个人信息； 2. 教师或学生用户登录系统，管理个人信息； 3. 教师或学生浏览或查询教材，需要的教材可加入书架，若查询的教材无库存可提交缺货订单； 4. 教师或学生订购教材并可在查看订单之后可确认领书，并对教材进行评价； 5. 教师或学生可查看教学计划，浏览公告，下载相关材料； 6. 教师用户可提交所需教材目录； 7. 退出系统。

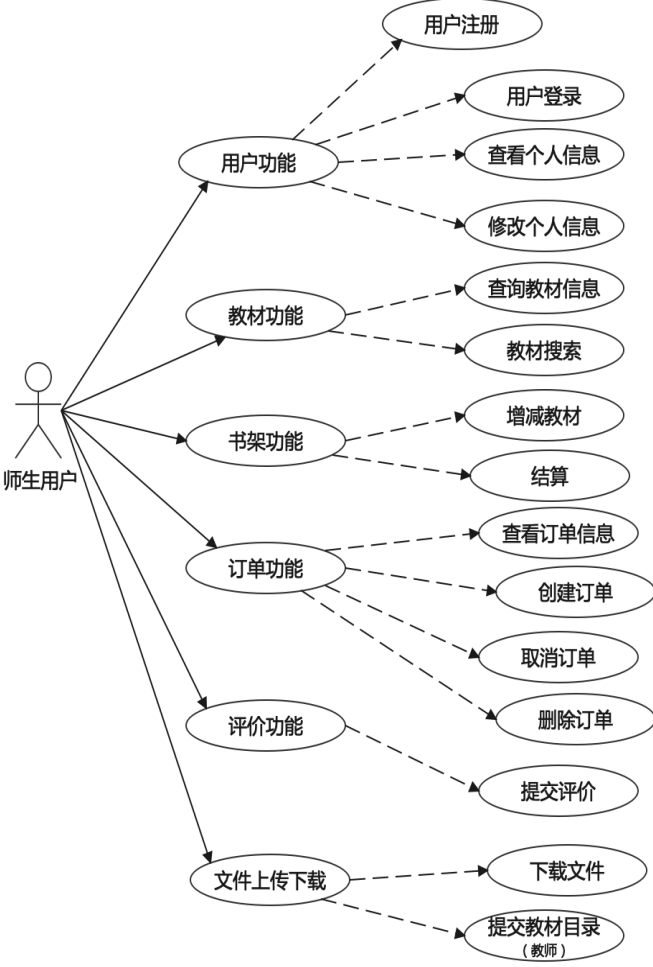


图 3.2 师生用户用例图

教材征订系统中,师生用户是指学校各系部的普通老师或者全体在校学生,他们有查询、订购、领取各门科目教材的需求,是教材管理工作的主要对象。师生用户注册登录之后就可以进入教材征订系统,能够对个人账户进行管理,例如对账号和密码等个人信息进行修改;对公告通知进行浏览,查看教材目录;根据自身需求,查询、订购和评价教材。教师也可根据自己的教学任务,提交相应的教材目录材料。

3.2.2 教材科用例

教材科属于管理人员,需要完成教材入库,教材订单汇总,用户信息管理等功能。从教材科的角度,整理得到用例规约如表 3.2 和相应的用例图 3.3。

表 3.2 教材科用例规约

项目	内容
用例名	教材科用例
参与者	教材科管理人员
描述	教材科管理人员通过管理员预先设定的账号登录系统，管理个人账户信息以及添加管理用户；在系统中审核订单后批量订购教材并进行订单汇总；教材收货后对教材进行入库管理，学生领书后管理相应库存；还有基本的浏览、查询教材、添加删除教材、修改教材信息，浏览教学计划，下载材料等。
前置条件	登录成功，存在当前任务。
触发	教材科触发
后置条件	教材查询、订购、教材信息库存变化、用户增减、用户评价信息导入
过程	1. 教材科管理人员登录系统，查询信息，增减用户； 2. 查看教学计划，浏览公告，下载相关材料； 3. 浏览查询教材，增减教材，编辑教材分类，修改教材参数信息，教材收货后对教材进行入库管理，学生领书后确认出库，更新教材库存； 4. 根据教学计划，通过审核的订单，提交批量教材订购订单，汇总订单； 5. 导入师生需要评价的对应教材信息，查看评价结果； 6. 退出系统

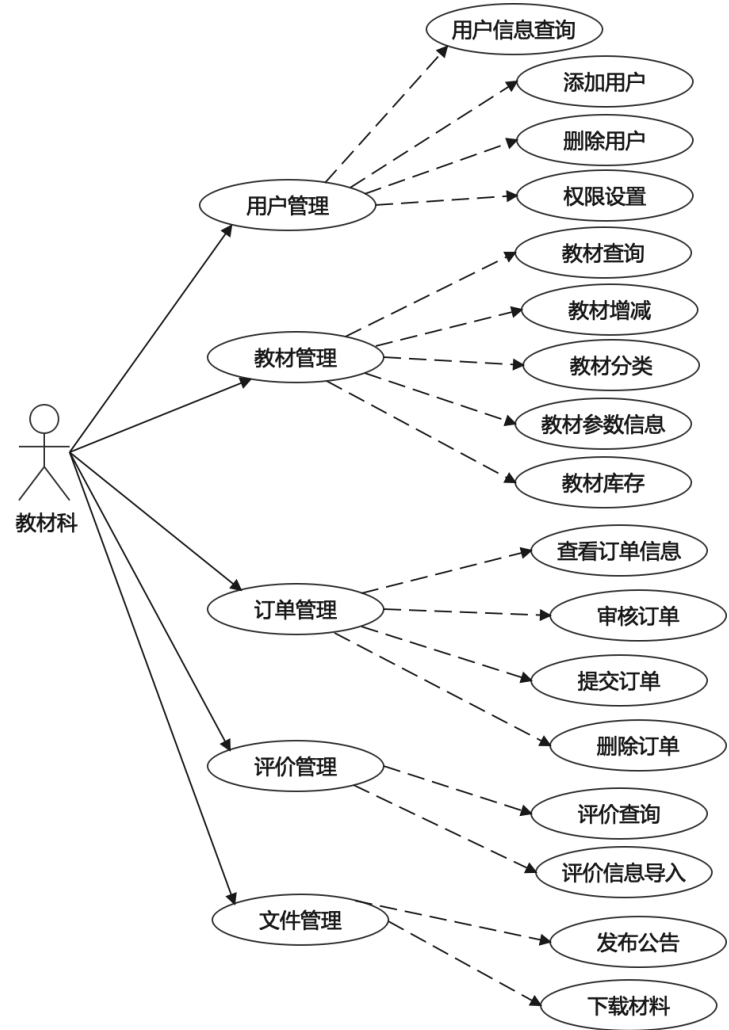


图 3.3 教材科用例图

在教材征订工作中，教材科是保证教材征订工作进行的管理角色，一般由科室中的管理人员和日常事务处理的科员担当。此类角色的用户凭借预先设定的账号登录系统，(1)管理个人账号信息，增减用户以减少数据冗余；(2)增减系统中的教材，对教材进行分类、修改参数信息等；(3)查看教学计划，并查看相应教材库存；(4)根据系部提交的教材订单，结合教学计划和教材库存进行审核，进行教材批量订购并汇总订单；(5)对新订购的教材到货后进行相应的教材入库处理，待学生领完书后更新领取状态，确保库存的及时更新；导入每学期师生需要评价的对应教材信息，查看评价结果。

3.2.3 教务处用例

教材征订系统管理人员也包含教务处，主要负责教学计划发布和管理、相关公告发布和用户管理等。根据整理的教务处功能需求，制作成表 3.3 用例规约，图 3.4 是相应的用例图。

表 3.3 教务处用例规约

项目	内容
用例名	教务处用例
参与者	教务处管理人员
描述	教务处管理人员通过管理员预先设定的账号登录系统，管理个人账户信息以及添加管理用户和出版社用户；在系统中发布管理教学计划，以及日常公告的发布与管理；还有对教材科提交的教材订单审核，提交最终订单。
前置条件	登录成功，存在当前任务。
触发	教务处触发
后置条件	教学计划的发布，提交最终教材订单
过程	1. 教材科管理人员登录系统，管理个人信息，增减用户； 2. 发布教学计划、日常公告； 3. 对教材科提交的教材订单进行审核，确认无误后提交； 4. 退出系统

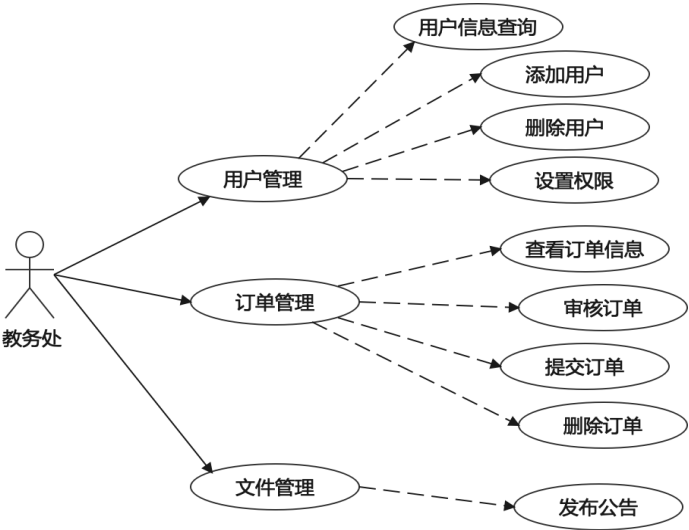


图 3.4 教务处用例图

在教材征订工作中，教务处是保证教材征订工作进行的前提角色，一般科室中的管理人员和日常事务处理的科员来担任教务处的管理人员。他们通过之前注册的账号登录进系统，对个人账号信息进行管理，对出版社账户进行增减操作；在每学期快结束时，公布下学期的教学计划，并对教材科上报的教材订单进行审核，审核通过后提交供出版社查看，方便新学期教材征订工作的及时开始。

3.2.4 系部用例

系部主要是根据教师上传的教材目录，最终确定提交本系部教材订单。从系部的角度，整理得到用例规约如表 3.4 和相应的用例图 3.5。

表 3.4 系部用例规约

项目	内容
用例名	系部用例
参与者	系主任
描述	系主任通过管理员之前注册的账号登录进系统，对个人账户信息进行管理；在系统中查看老师上传的教材目录；上传本系部教材订单给教材科审核。
前置条件	登录成功，存在当前任务。
触发	系部触发
后置条件	系部订单通过审核，系部教材目录通过审核；
过程	1. 教材科管理人员登录系统，管理个人信息； 2. 审核教师所上传的教材目录； 3. 上传本系教材订单； 4. 退出系统

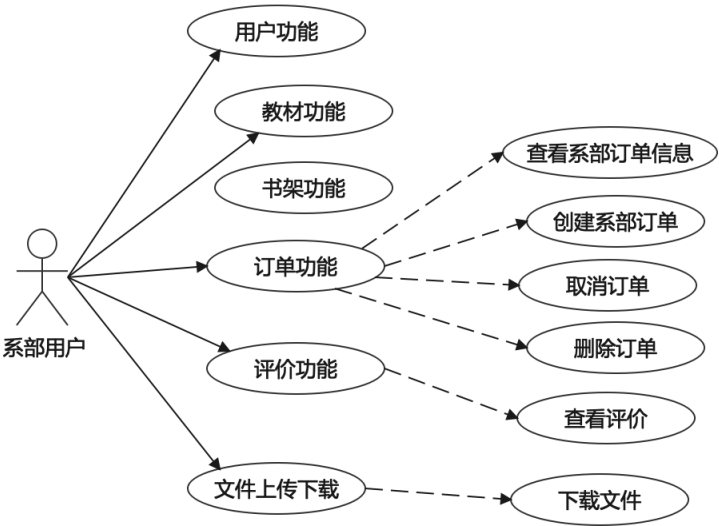


图 3.5 系部用例图

在教材征订工作中，系部是保证教材征订工作进行的重要角色，一般由系主任担当。他们通过之前注册好的账号登录系统，对个人账号信息进行管理，具有普通用户功能；在每学期末，查看老师上传的下学期课程所需教材的目录，确认后对本系部所有课程所需教材进行

汇总整理，最后上传本系部新学期所需要订购的教材订单；查看教材评价作为日后订购教材的参考。

3.2.5 出版社用例

出版社主要查询学校提交的订单，对教材进行入库。从出版社的角度，整理得到用例规约如表 3.5 和相应的用例图 3.6。

表 3.5 出版社用例规约

项目	内容
用例名	出版社用例
参与者	出版社人员
描述	出版社人员通过管理员预先设定的账号登录系统，管理个人账户信息；在系统中查询教材订单；被授权后可进行教材入库。
前置条件	登录成功，存在当前任务。
触发	出版社触发
后置条件	完成教材订购，教材库存更新
过程	1. 出版社人员登录系统，管理个人信息； 2. 查询教材订单； 3. 被授权后可进行教材入库； 4. 退出系统

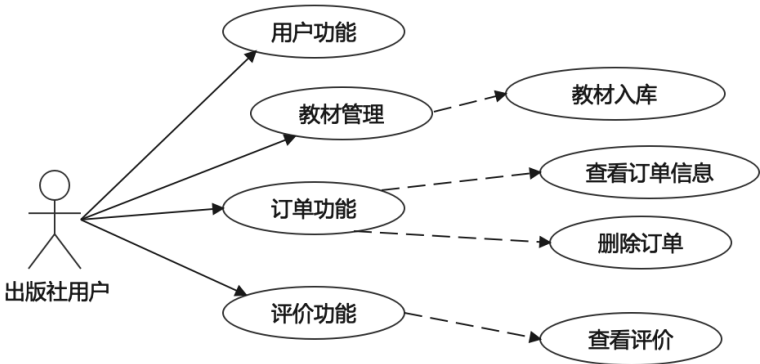


图 3.6 出版社用例图

在教材征订工作中，出版社是教材提供方。他们通过之前注册的账号登录系统，对个人账号信息进行管理；查询订单，根据订单提供教材；提供教材后在被授权的情况下可完成教材入库；可查询出版社教材评价结果。

3.3 系统非功能需求分析

系统需求分析过程中，除了功能性需求，还有非功能性需求，这样才能保证系统更好的服务于教材征订工作，服务于全校的师生，性能要求主要有以下几个方面：

（1）可用性

确保系统的微服务任何时候都能够运行，保证其有一个很强的可靠性。个别微服务发生故障时不影响其他微服务的运行，使得系统在任何情况下都能够运转。

（2）并发性

根据教材征订系统的特性，考虑到普通大学的在校人数，每学期全体师生需同时选课并选用相关教材、需同时对本学期教材进行评价以及系统的未来发展，系统应该具有很强的并发能力，而且不能有过长的响应时间。即使用过程中师生等待加载页面的时间，或者与系统进行交互的时间不能太长。

（3）可扩展性

教材征订系统的功能随着科技的发展将逐渐增加、完善。业务也在不断变得复杂起来，系统应该追求降低添加新的微服务对整体系统的影响，故而为了应对未来的快速更新，本文的系统应该具备很强的可扩展性。

（4）可维护性

一个好的编码习惯、十分详细的文档说明还有测试的复杂程度都对后期维护起着十分关键的作用，由于当今时代的发展，本文的系统必定要不断的更新以及维护，所以系统要具备一个很好的可维护性来方便之后的工作。

（5）安全性

由于信息安全的需要，本文的系统对于用户信息安全要求很高。主要从系统安全以及数据库安全两个方面进行考虑，如果系统数据安全出现了问题，损失将不可估量。在这种情况下，有必要确保系统的安全性。

3.4 本章小结

本章针对教材征订系统的需求进行了分析介绍。首先详细介绍了本文系统的目标要求与角色组成。然后从用户角色的角度为系统设计了五个用例，通过用例规约构建了用例图对系统的需求进行了描述，并确定了教材征订系统应有的功能，绘制了教材征订系统的功能结构图，同时对系统的非功能性需求也提出了要求。

第四章 教材征订系统设计与实现

本章对基于微服务的教材征订系统的总体设计进行了介绍，区分了教材征订系统的服务，设计了教材征订系统的微服务架构实现方式，以 Spring Cloud 为基底设计实现微服务基础组件，包括 Eureka 注册中心组件、Zuul 网关组件、Config 服务配置管理组件以及各功能模块具体设计，并对所设计系统的高并发性能进行了分析。最后使用流程图、类图和时序图对各个微服务进行详细说明，并结合相应的数据库设计来完成分布式教材征订系统的搭建。

4.1 基于微服务的教材征订系统总体设计

经过对教材征订系统需求进行分析，为了方便后期扩展与维护，教材征订系统采用微服务架构进行开发，其整体架构设计如图 4.1 所示。

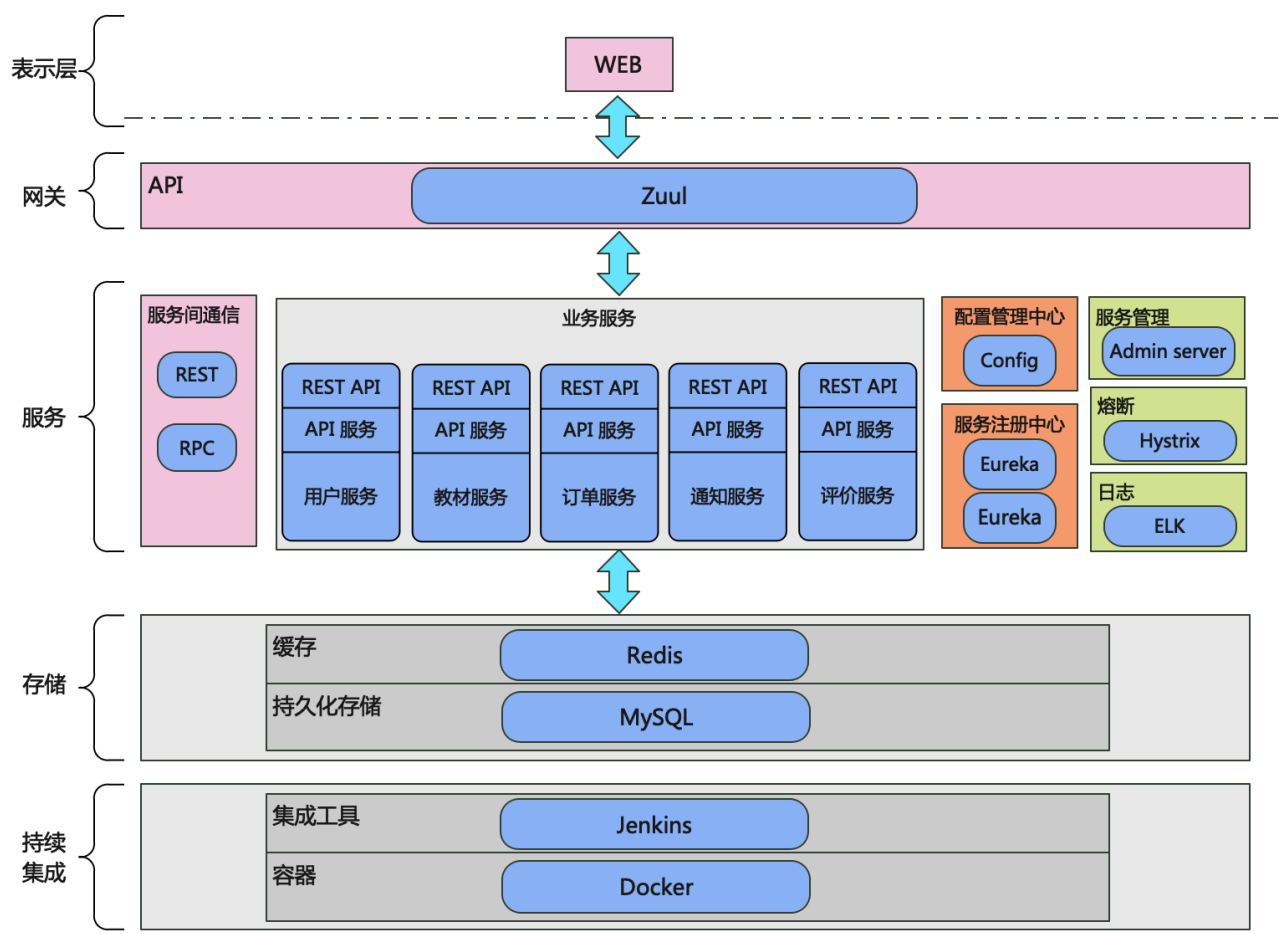


图 4.1 系统架构图

本系统主要包含表示层、网关层、服务层、存储层，其中服务层是核心，采用微服务架构。表示层负责和用户进行交互，包括 WEB 页面等。网关层是系统的统一入口，外部通过

统一的 API 网关接入所需要调用的微服务，同时也可以处理一些非业务功能，如监控、负载均衡和身份认证等。Zuul 有验证用户身份功能，审核用户发来的信息，按照相应的地址路由转发到相应的微服务。如果不使用 Zuul，就会造成代码冗余跟无法保证用户隐私安全的问题。

服务层是整个教材征订系统最核心的部分。服务层基于 Spring Cloud 框架构建，其中业务服务包括用户服务、教材服务、书架服务、通知服务、订单服务、评价服务共六个部分。REST API 是一套对多种前端跟一个后台交互方式进行规范的协议，Rest 是 REST API 接口规范，对外提供 Http 接口，用来接收并响应 Web 端的请求接口。API 服务就是实现 REST API，进行参数预处理，包括参数校验、参数规格化等动作，然后将请求转发到微应用上；在收到微应用响应后，构造响应体返回。REST API 和 API 服务是同一层，声明接口规范并对网关层使用 RestHttp 进行通信，从而达到高内聚低耦合的效果，对内微服务之间的通信方式是调用 RPC。服务层全部的服务和 API 网关都注册在了 Eureka 里面，不需要特定的 IP 地址进行相互调用。同时利用 Hystrix 进行容错处理，再配合 Config 统一的管理服务配置，架起一个十分完整且功能比较齐全的体系框架。本系统设计的服务层的逻辑架构如图 4.2 所示。服务网关、服务消费者、服务提供方集群、配置中心集群都注册在服务注册中心集群。外部则通过统一的 API 网关接入所需要调入的微服务。

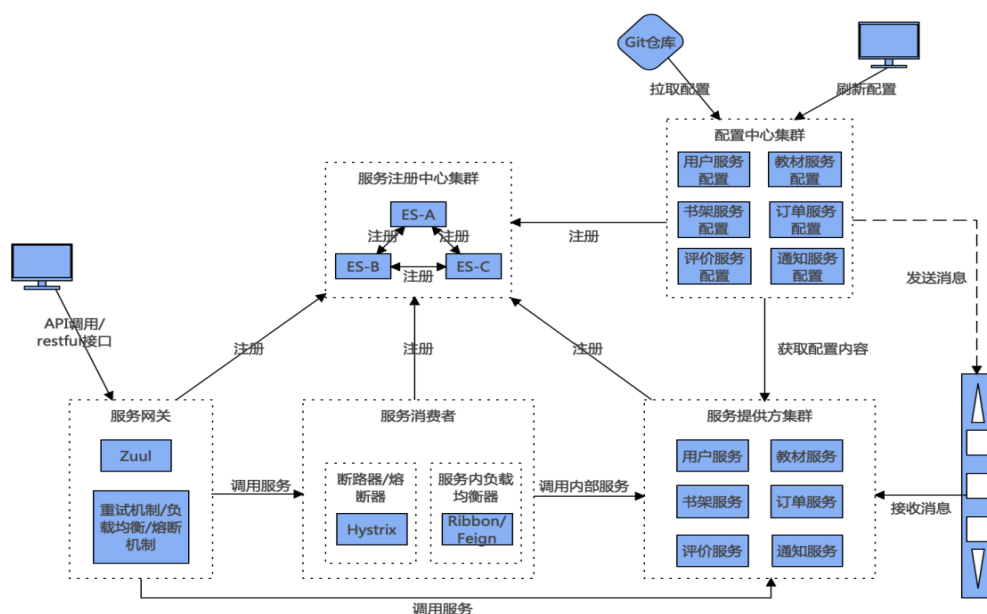


图 4.2 服务架构图

存储层需要持久化存储数据以及对数据持久化缓存，服务层里的每个模块提供服务给表示层的时候，肯定会从存储层对数据进行读取。系统里用 MySQL 数据库来实现结构化数据，图里的数据库一般被当作是数据库管理系统，分为多个数据库来对应不同的业务，每个服务对应一个能够独立布置的单独的数据库。请求量比较大的数据缓存在 Redis 中，服务可以直接访问 Redis 缓存，提高数据获取效率。

Jenkins 通过拉取项目源码，编译并打包，然后构建成 Docker 镜像。Jenkins 还可以发送远程命令，让生产部署服务器拉取镜像到本地，然后创建容器。最后用户可以访问到容器。

4.2 服务组件设计与实现

从本质上来看微服务系统属于分布式系统，也要对一些组件或相应的服务进行设计，搭建一个适合分布式的微服务环境。SpringCloud 提供了丰富的组件可以让用户能够很快的搭建一个微服务架构，本文主要使用 Eureka 组件实现服务的发现与注册，通过 Zuul 组件来构建本系统的网关服务，Spring Cloud Config 则实现微服务的外部化配置管理。

4.2.1 服务注册与发现设计与实现

服务注册中心在微服务架构中特别关键，因为一般是多个微服务组成一个分布式的微服务系统，本文中的系统就由六个微服务组成，需要通过服务发现组件来发现与注册服务从而启动服务中的多个实例。本文设计使用服务注册与发现组件 Eureka，如图 4.3 所示，在启动系统的每个微服务时，都要在 Eureka 中注册自己的网络地址信息，服务提供者的网络地址就能通过服务发现组件被服务消费者发现，这样服务提供者的接口将根据这个地址被调用。由于微服务与服务发现组件之间特有的通信机制，服务发现组件会注销长时间无响应的微服务。如果微服务网络地址有变化发生的时候，会在服务发现组件里进行重新注册，服务消费者不需要手动对服务提供者的网络地址进行修改。

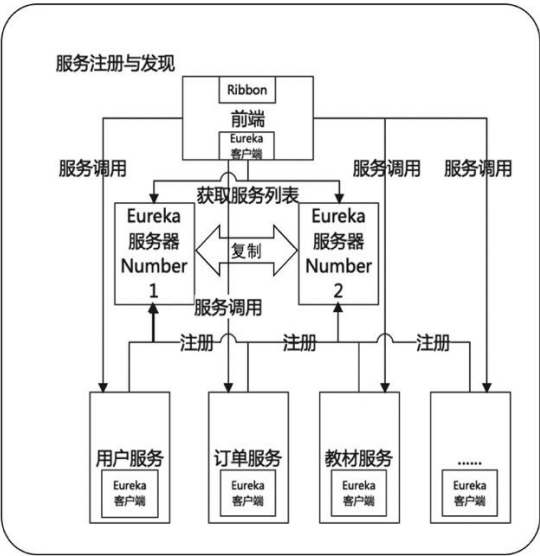


图 4.3 教材征订系统服务注册与发现

图 4.3 主要列出本系统的三个微服务，用户管理微服务、订单微服务以及教材微服务等；

如图所示：首先 Eureka 服务器收到 Eureka 客户端进行的注册并且进行存储，这些内容将提供给前端调用。最后由 Eureka 服务器在 Eureka 客户端发现服务后完成注册服务的工作。

Eureka Server 端通过对 Server 依赖进行添加、修改配置后在配置文件里添加注解就能实现配置注册中心，从而实现服务注册与发现。首先添加 Eureka 依赖到 pom.xml 文件里，具体代码如下：

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
  <version>2.2.0.RELEASE</version>
</dependency>
```

添加好 Server 依赖后，在配置文件 application.yml 中修改参数。具体代码如下：

```
server:
  port: 9001                //注册中心端口号
eureka:
  instance:
    hostname: localhost    //注册中心实例的主机名称
client:
  fetch-registry: false    //是否提供检索服务
  register-with-eureka: false //是否向注册中心注册自身
  service-url:
    defaultZone: http://${eureka.instance.hostname}:${server.port}/eure
                        //设置与 Eureka Server 的交互地址
```

完成配置之后，添加@EnableEurekaServer 注解到启动 Spring Cloud 的入口类上，配置完成注册中心。Eureka Client 端跟 Server 端配置大致一样，同样要对 Client 依赖进行添加，如果一个配置文件被修改之后只需要添加注解就好。Eureka Client 跟 Server 端中的端口不一样，到这完成全部的服务注册与发现步骤。

系统功能具有一定的特殊性，要尽量保证系统的高可用性，故通过若干节点互相进行注册的方法对注册中心 Eureka 达到集群高可用的目的。如图 4.3 所示，Eureka 服务器 Number1 与 Eureka 服务器 Number2 是两个 Eureka Server 实例，Number1 和 Number2 实例共同构成了一个集群，高可用部署是通过相互注册的方式实现的，这两个实例同步信息是通过相互以增量的方式实现的，对数据的一致性起到确保的效果。每个微服务中都配置 Eureka 客户端，客户端要跟 Eureka 服务器交互运行，客户端还可以缓存数据，服务消费者能够在服务器宕机的时候在缓存里寻找到服务提供者并且对服务进行调用。

4.2.2 服务网关设计与实现

通过前文系统功能模块的划分，我们按需求将总系统功能划分为六个业务服务，各微服务和网关之间的架构关系如图 4.4 所示。只有先通过网关，前端才能到达后端的微服务进行访问。

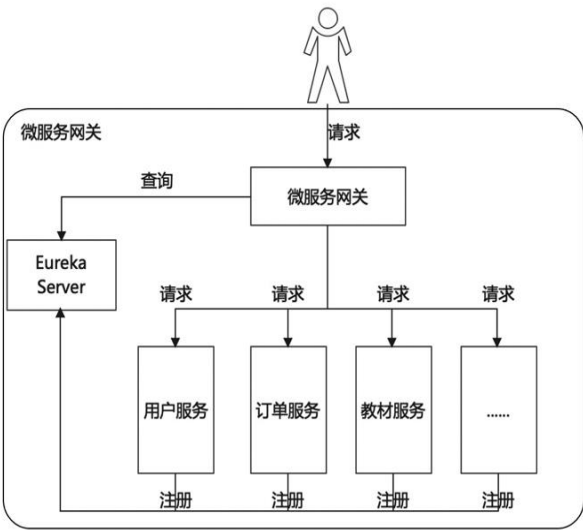


图 4.4 教材征订系统服务网关

本系统的网关服务是通过 Zuul 组件框架来实现的，同样将它设计成一个被注册到服务注册中心的能够独立运行的微服务，步骤一般如下：

（1）网关服务依赖：在父工程中创建一个名为 ZuulServer8001 的子模块，通过导入 Zuul 依赖、服务配置中心依赖和服务注册中心依赖使网关服务必须关联服务注册中心和服务配置中心，因为 Zuul 本身对 Ribbon 和 Hystrix 进行了整合，所以不需要二次导入依赖。

（2）网关服务配置：网关服务相关的配置文件存放在配置中心，本地配置文件一般命名为 bootstrap.yml 以负责关联注册中心和配置中心，放置在配置中心的 ZuulServer- dev.yml 文件，则负责配置网关服务的端口号以及服务注册：

```
server:
  port: 8001
eureka:
  client:
    service-url:
      defaultZone: http://localhost:9001/eureka/
```

（3）网关整合容错：Hystrix 包含所有转发的网关的请求，且 Hystrix 是 Zuul 默认整合的，用以监控微服务。在现实运用中，许多业务功能往往要靠一个微服务系统完成，这个微服务系统具有许多服务层，一个业务请求的实现还有可能涉及多个服务间的相互调用。服务 A 被

服务消费者调用，服务 B 被服务 A 依赖，服务 C 被服务 B 依赖，这一个完整的调用链成功响应的前提是依赖服务依次正确返回响应的结果。只要其中一个服务有错误或者响应时间太长，便会影响整条调用链的正常运作，从而发生雪崩效应。因而要设计一个十分强大的容错机制给微服务系统来防止雪崩效应。

Hystrix 可以起到断路保护的作用，整个分布式系统可能会因为个别微服务访问延迟或者访问量十分巨大这些问题使得暂时不能提供服务，当这种现象出现后，Hystrix 则会发起熔断请求，然后返回错误信息，从优化系统稳定性和服务资源占用比率方面提升了系统运行效率。系统前端在调用用户管理微服务、教材微服务和订单微服务时，都配置上 Hystrix 熔断器。

(4) 启动网关服务：通过使用 `@SpringBootApplication` 注解和 `@EnableZuulProxy` 注解设置网关服务启动类，来完成 Zuul 代理启动。之后通过 `http://{zuul_host:zuul_port}/{service_id}/**` 的格式对接口进行访问，发送端的请求就会通过网关然后再被转发到相应微服务的接口。Zuul 网关服务之所以成为一个能够独立运行的服务，只需在注册中心对其注册，就能够开启若干网关服务实例，使得网关服务具有很高的可用性。

4.2.3 配置中心设计与实现

考虑到本系统为微服务系统并且是分布式的，其中有六个服务，每一个服务还能够启动多个实例，这就需要有一个集中管理功能对微服务配置进行管理。如图 4.5 所示，Spring Cloud Config 就提供服务端和客户端来实现微服务的外部化配置管理。每一个微服务的 `application.yml` 配置文件被配置中心使用 GitHub 托管，来达到统一管理的目的，集中式的管理所有微服务中的配置文件，实现配置环境的动态更新，并且提供不同的配置给不同的环境。不需要重启微服务来应对微服务业务需求变化带来的配置文件的修改，配置文件动态更新的实现，对于系统的维护与开发测试提供了很大的便利。

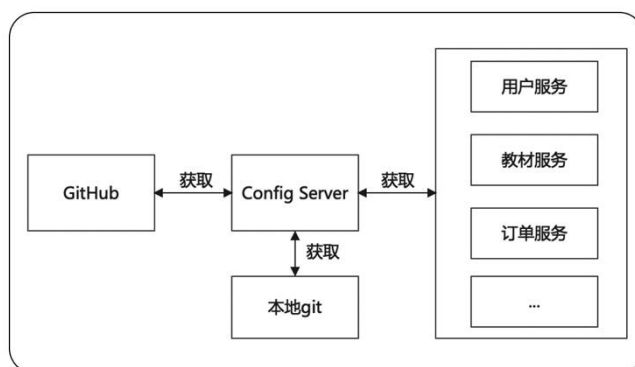


图 4.5 教材征订系统分布式配置

如图 4.5 所示,所有服务需要在注册中心注册,并且在配置中心 Config Server 的 shareConfig 目录下统一存放每个服务的配置文件,每个服务的配置文件的命名方式统一为{服务名}+{-dev/prod}+{文件后缀名 yml},用以说明这些微服务配置文件作用于开发环境或生产环境。

(1) 配置中心依赖:使用 Spring Boot 在父工程 JudgementSystem 下创建子模块并且起名为 ConfigServer8002 当作配置中心服务,同时引入 ConfigServer 依赖在依赖管理 pom.xml 文件里:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-config-server</artifactId>
</dependency>
```

(2) 启动配置中心:配置中心服务虽然是能够独立运行的微服务,也一样要对配置中心服务的启动类进行编写并运行,使用@SpringBootApplication 和@EnableConfigServer 注解就能启动自动化配置功能从而进行配置的自动刷新。

(3) 添加配置中心配置文件:

```
server:
  port: 10000
spring:
  application:
    name:config-sever
cloud:
  config:
    server:
      git:
        url:http://gitee.com/it-lemon/config-repostory.git
```

4.3 基于微服务的教材征订系统高并发性能分析

为优化系统在短时间内访问量激增而出现的高并发问题,本文通过构建分布式系统并采用负载均衡技术完成微服务架构。本小节通过对比微服务架构与传统单体架构处理大规模访问量的方式,以论述微服务架构通过负载均衡对系统高并发性能进行优化的可行性。

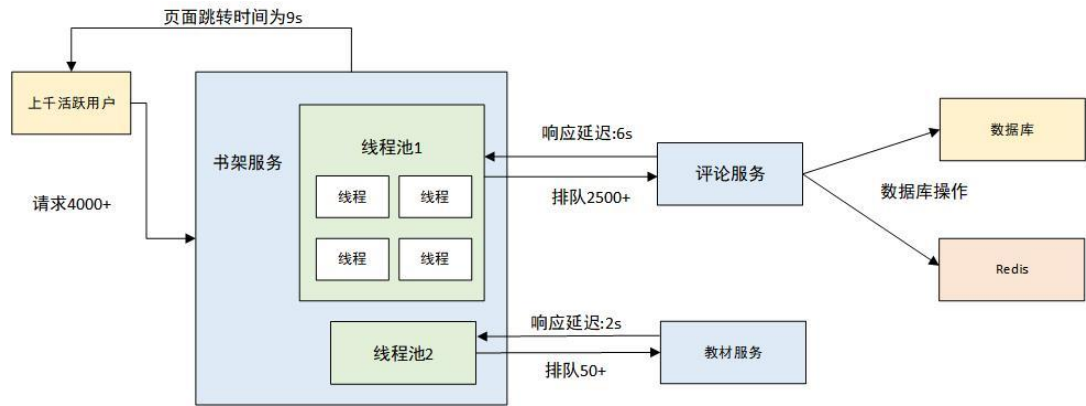


图 4.6 传统单体架构高并发服务器请求处理过程

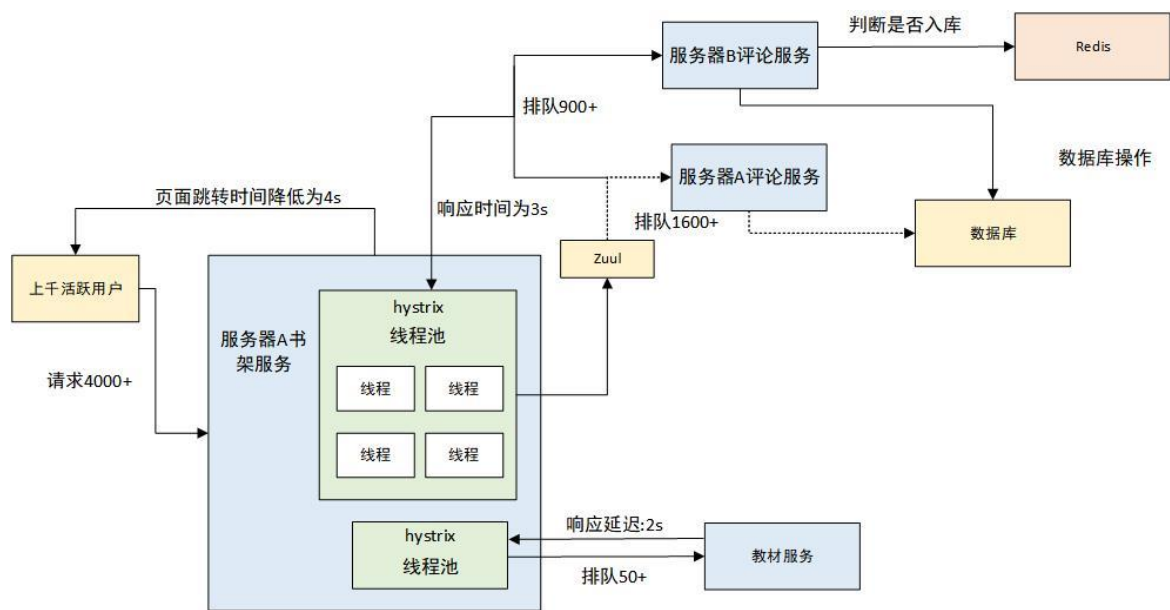


图 4.7 微服务架构高并发服务器请求处理过程

如图 4.6 所示，传统的单体架构在服务端上处理请求的过程一般遵循先来后到的顺序，当一段时间内出现大量的接口请求时，服务器只能依次对其进行处理，这时系统响应时间取决于服务器 CPU 的性能和算法复杂度，这样当系统缓存区域过大导致后续的请求被定义为超时响应而丢弃，从而在客户端处造成页面跳转不畅，或者出现请求错误页面。这种服务器负载过高的问题，正好可以通过微服务的灵活部署和负载均衡器进行优化。如图 4.7 所示 Zuul 网关将 Ribbon 负载均衡器内嵌在其中，当用户需要对教材书籍进行评论时需要调用书架微服务的图书查询接口，从书架微服务获取图书相关信息后，再跳转至评论服务，当其中一个评论微服务的接口接受到过多请求，网关则会根据负载均衡算法（本文采用加权轮转调度法）将请求分发到其它服务器中进行处理，这些服务器可以灵活地部署在校园各处，同时微服务

代码可以灵活部署于不同的主机上，通过软件和硬件结合的方式对短时间的高并发问题做出有效的优化，也提高了硬件资源的利用率。最终，处理完成后的数据会汇入微服务专属服务器中。因此微服务框架通过 Ribbon 负载均衡器将用户请求转移到不同服务器上处理，能大大降低请求的排队时间，在高并发环境下给用户带来更好的使用体验。

4.4 教材征订系统服务功能设计

微服务架构设计的首要切入点以及难点是微服务的划分问题，在微服务领域中有着十分关键的地位^[66]。但正是因为微服务的每一项服务划分没有固定和统一的标准，所以无法按照统一的思想或模版来进行微服务划分。总体上看系统架构螺旋向上的发展，微服务的设计应依据现实的业务功能，选择一个尽可能平衡的过程，在开发成本和系统性能间进行权衡，最大限度的提高应用系统的可扩展性和可维护性，从而使所开发的系统具备更长的生命力和更强的竞争力

通过对第三章系统整体功能的需求进行分析，本系统被划分为用户微服务、教材微服务、书架微服务、订单微服务、评价微服务、通知微服务这六个功能模块。整个系统划分服务后的功能模块图如图 4.8 所示。

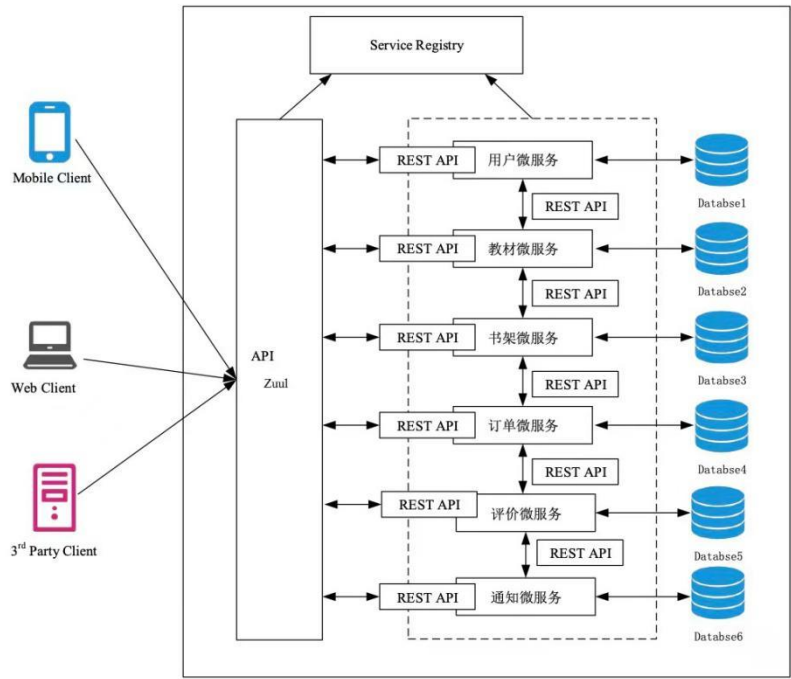


图 4.8 系统功能结构总图

4.4.1 用户服务功能设计

本文根据需求分析以及对具体实际进行分析，将用户微服务分成四个子模块，其中包含注册登录、个人信息管理、账号安全以及我的关注，其功能结构图如图 4.9 所示。

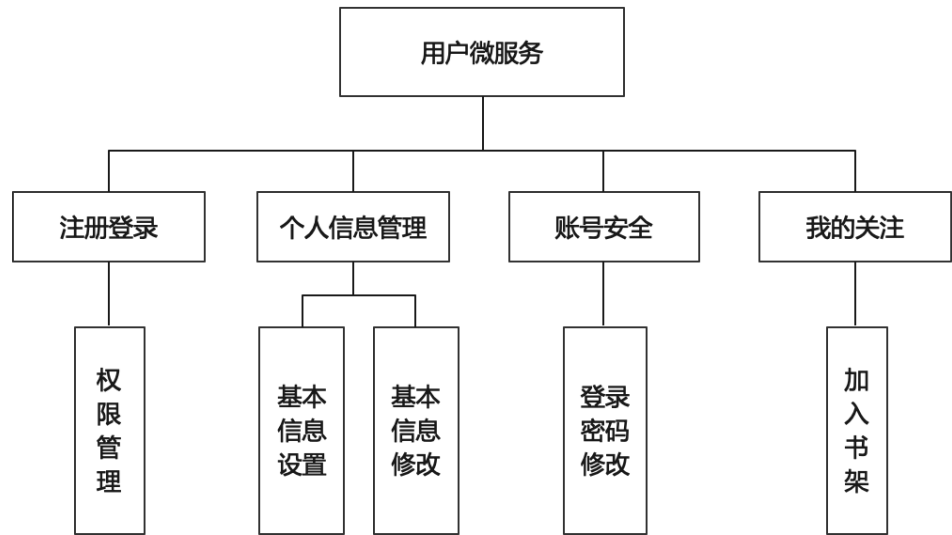


图 4.9 用户微服务功能结构图

- （1）注册登录：不同用户选择不同的使用权限注册后，通过审核，即可登录。
- （2）个人信息管理：对用户的个人信息进行管理，包括头像、手机号等信息，用户能够自己进行修改设置。
- （3）账号安全：提供登录密码修改功能和手机号更改或邮箱更改功能。
- （4）我的关注：关注想要的教材，决定是否加入书架。

4.4.2 教材服务功能设计

教材微服务主要负责与教材相关的所有管理。教材微服务被分为三个部分，包括教材管理、分类管理、教材参数信息，其功能结构图如图 4.10 所示。

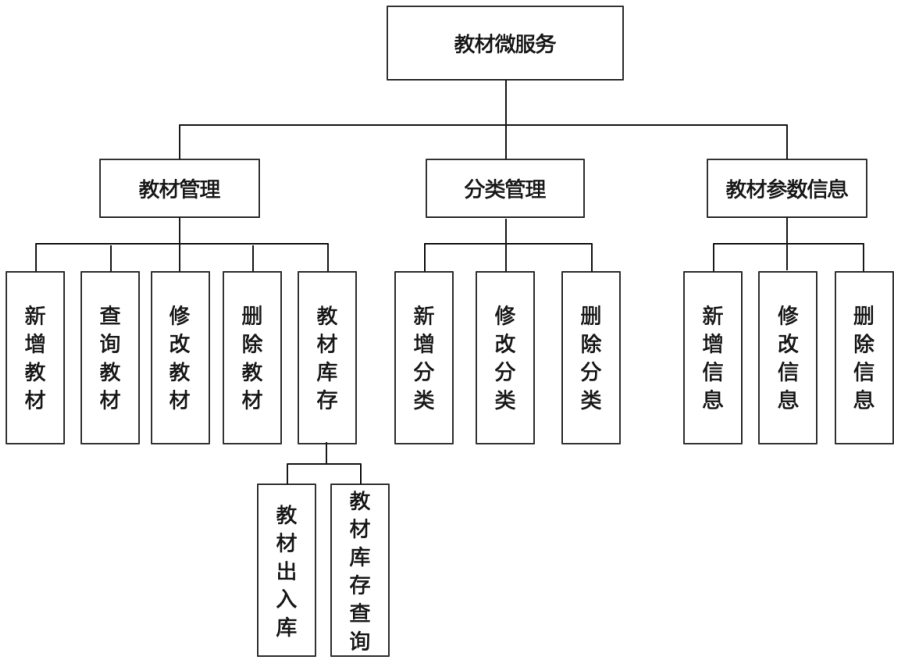


图 4.10 教材微服务功能结构图

- (1) 教材管理：教材管理员负责教材的查询、新增、删除、修改以及教材的库存。其中教材库存管理包含了教材的入库出库以及教材的库存查询。
- (2) 分类管理：教材管理员对教材类别进行新增修改，也能够对已有的教材类别进行删改。
- (3) 教材参数信息：管理员添加教材的同时也要添加教材的参数信息，这是两个互相关联的模块，以教材管理为基础进行教材的参数信息管理，可以新增教材的信息，修改已存在的的信息，删除不必要的信息数据。

4.4.3 书架服务功能设计

这个微服务是系统给用户提供的放置想购买的教材，有对教材数量进行修改，对不需要的教材进行删除，对教材进行结算这四个功能，其功能结构图如图 4.11 所示。

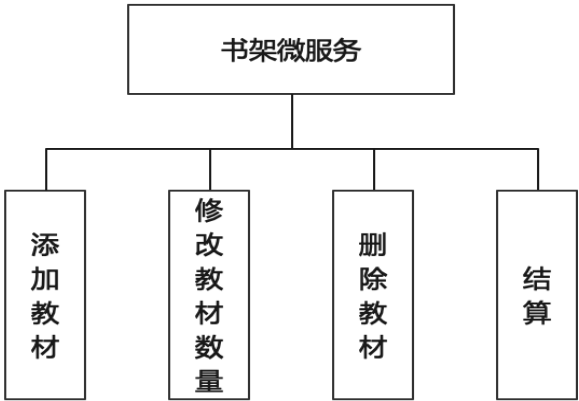


图 4.11 书架微服务功能结构图

- (1) 添加教材：用户浏览教材时，遇到需要的教材可添加到书架中，最后统一创建订单。
- (2) 修改教材数量：用户根据自己需求选择购买教材数量。
- (3) 删除教材：用户根据自己需求删除不需要的教材。
- (4) 结算：用户选定教材后进行结算，跳转到订单页面。

4.4.4 订单服务功能设计

订单微服务主要管理订单的操作，主要功能为：创建订单、查询订单、取消订单、删除订单、审核订单和订单评价，如图 4.12 所示。

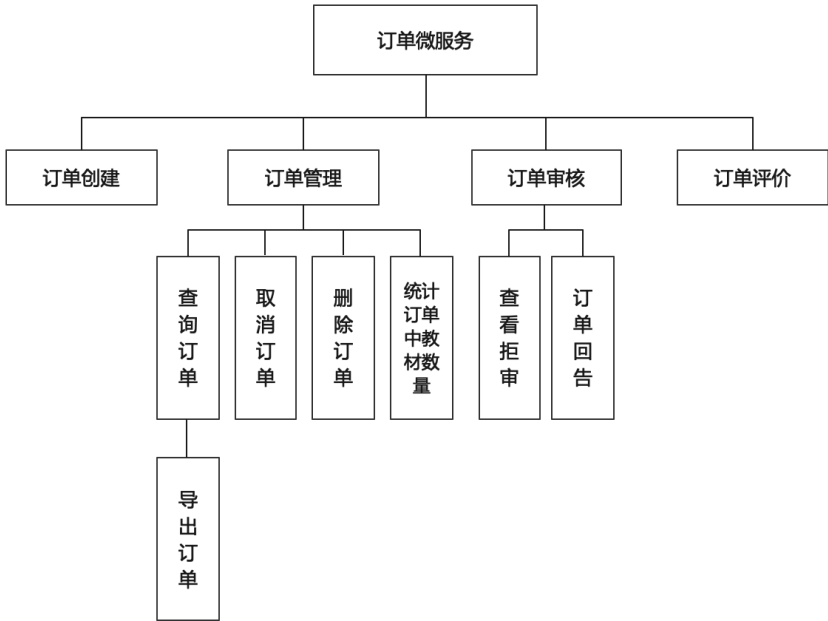


图 4.12 订单微服务功能结构图

- (1) 订单创建：当用户进行教材结算时，跳转订单创建子功能。
- (2) 订单管理：用户可以查询订单，取消订单，删除订单，后台会统计订单中教材数量确保教材库存准确。在用户进行订单查询时，可将订单导出。
- (3) 订单审核：管理员可以对订单进行审批，用户可查看到拒审订单。对于无法征订的教材反馈订单回告。
- (4) 订单评价：当用户完成订单后，对订单进行评价。

4.4.5 评价服务功能设计

评价微服务主要负责整个系统对教材的评价，所以将其单独设立为一个服务。主要是每学期结束后，全体老师和学生都要对本学期所授或者所学的教材进行相对应的评价。评价结果会反馈给教务处和出版社，便于后续教材征订的调整。主要包括评价提交、评价查询、

信息导入三个功能，其功能结构图如图 4.13 所示。

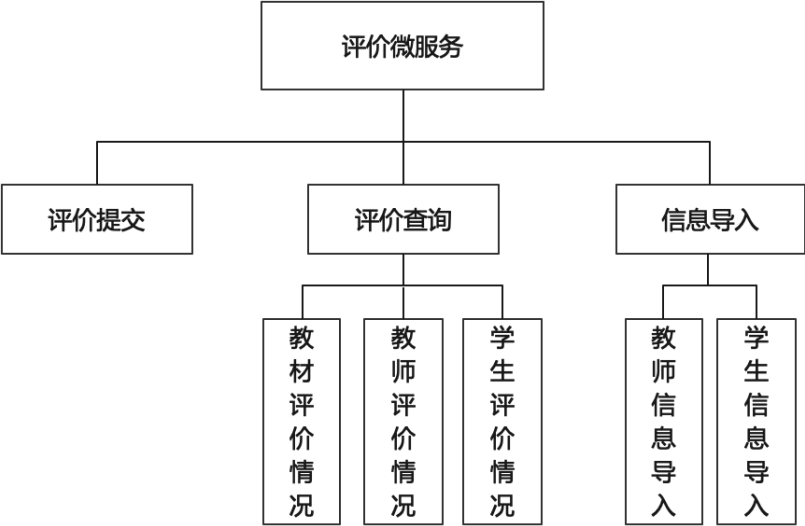


图 4.13 评价微服务功能结构图

- （1）评价提交：当用户对订单教材进行评价后，可提交评价。
- （2）评价查询：管理员可以对教材的评价结果进行查询，可查询具体教材，某个老师，某个学生的评价情况。
- （3）信息导入：管理员可以将本学期使用教材的教师和学生信息导入系统，以便师生用户显示本学期所需评价教材。

4.4.6 通知服务功能设计

围绕通知而进行的业务功能，其中包括管理通知信息以及文件的上传下载，其功能结构图如图 4.14 所示。

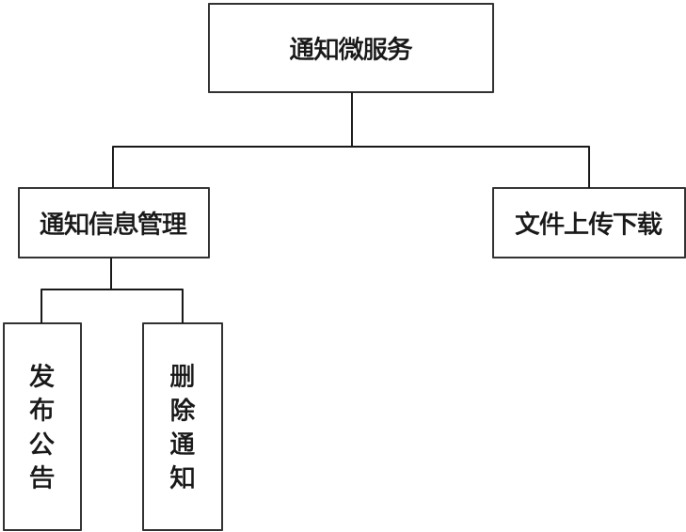


图 4.14 通知微服务功能结构图

- （1）通知信息管理：管理员进行公告（教学计划、通知等）的发布和删除。

(2) 文件上传下载：系部需要通过通知按时提交下学期教材订单，教师学生都要通过教学计划准备领取相应的教材和每学期的教学、学习准备工作，所以需要相关文件上传和下载功能。

4.5 教材征订系统业务服务的详细设计与实现

教材征订系统后台所涉及的功能实体类图如图 4.15 所示。教材征订系统的后台功能类图有 Admin、User、Books、Order、Notification 与 Comment 这六个大类。管理员的实体类就是 Admin 类，管理员登录上后台系统后，进行管理用户、教材、订单与通知等，管理员是后台系统已经设置好的用户，因而不需要对应的功能。用于教材征订的用户类是 User 类。管理员可以通过对用户的操作在后台查看用户信息，这个功能包含在 User 类里。要想对用户的具体信息进行查看只需点击对应用户就行，例如用户名、用户 ID、用户当前状态、用户邮箱等信息。用于管理教材的书籍类是 Books 类，可以用于对书籍信息进行查询；还可以对书籍列表进行查询、对书籍进行新增跟删除、对书籍进行审核、对书籍信息进行修改等。用户参与征订教材的订单类是 Order 类，可以在后台系统中对订单信息进行查询。Notification 类在后台的功能有管理参与和教材有关的各项通知，管理员可查看教材征订信息，例如教材征订时间、教材评价时间等。Comment 类在后台中可上传需要参与评价的用户信息，查看评价结果等。



图 4.15 教材征订系统后台功能实体类图

4.5.1 用户服务详细设计与实现

本节详细介绍了本文的用户功能模块设计。用户功能模块统筹管理了全部用户注册、登录、修改个人信息和对书架进行查看。下面以注册登录和修改个人信息为例，详细阐述流程设计。

(1) 用户流程设计

用户服务主要设计目的是让不同的使用教材征订系统的用户得到与其相对应的权限。不同的用户群在征订系统里拥有的权限不同。以教师和出版社权限为例，其中教师权限可以浏览、查询、订购教材，同时还可以使用提交教材目录、查看教学计划等功能。出版社权限主要用于教材入库操作。因此，为了方便管理，要制定不同的权限给不同的用户群。当用户注册成功以后，其注册信息将会被提交给后台系统审核。然后，教务处在用户信息管理模块里面审核新注册用户提交的注册信息，决定是否通过注册。通过注册用户审核流程，确保了进入系统角色可信度，从而保证了系统的安全。

用户登录系统时，系统将根据用户登录所选类型，通过查表操作查询用户的注册信息，待查找到匹配项后，开始提取该表中的该用户的登录权限标识符并进行身份认证。认证成功后将登录权限标识符存入公共类的静态变量中，然后运行该用户权限对应的主界面。其流程图如图 4.16 所示。

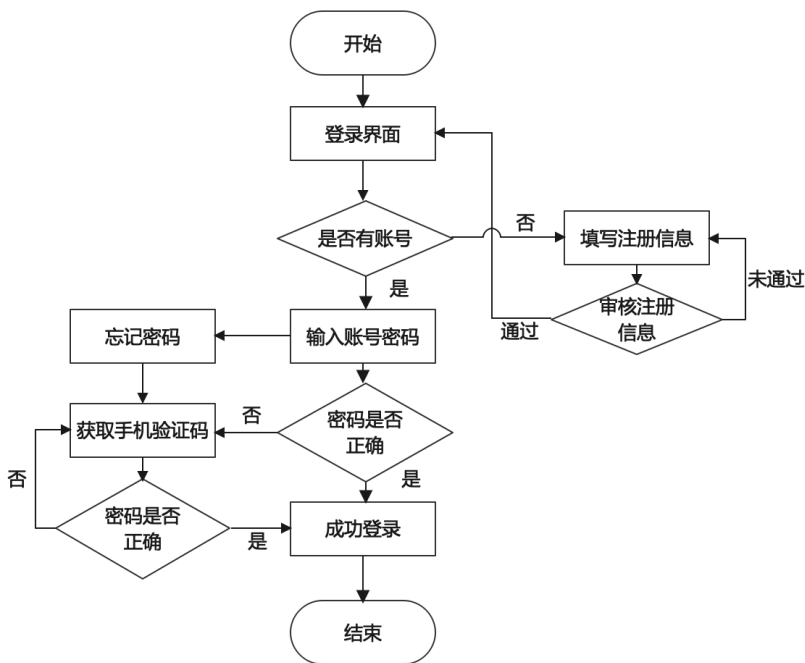


图 4.16 用户注册登录流程图

账号安全页面上要有修改密码和更改手机号的功能。手机号或邮箱其中之一在更改密码的时候要被保证有效。如果用户修改个人密码，则需要提供原密码。系统要求用户输入新密

码跟确认密码用于修改密码。在输入新密码和确认密码过程中系统将不停地校验新密码和确认密码是否相同，并即时给出提示。当用户要对修改密码进行确认时，要填写系统规定的信息，否则重新返回填写流程。其流程图如图 4.17 所示。

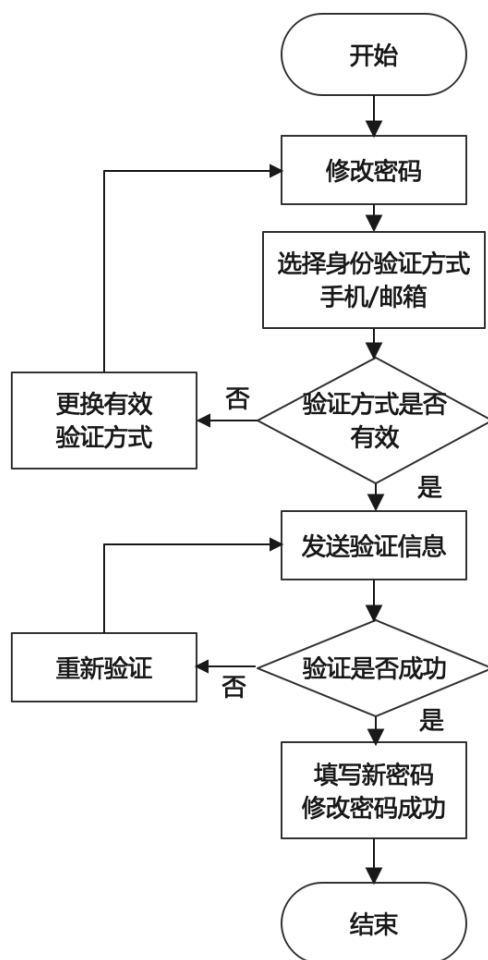


图 4.17 修改密码信息流程图

(2) 用户类设计

根据 Spring 框架的要求将各模块分为接口层、业务层与存储层三个层次。接口层主要是接收客户端的请求，它的 TextbooksUserController、LoginController 与 RegisterController 都是从 BaseController 继承的，对用户注册进行控制要用到 RegisterController，对用户登录进行控制要用到 LoginController，对查看、修改用户信息等进行控制需要用到 TextbooksUserController。Textbook-UserService 则是用作对增添新用户，用户登录、查询与更新用户信息等功能进行编写，它位于业务层并且是一个实现类。TextbooksUserDao 则被用于与数据库进行数据交互。用户服务的类图如图 4.18 所示。

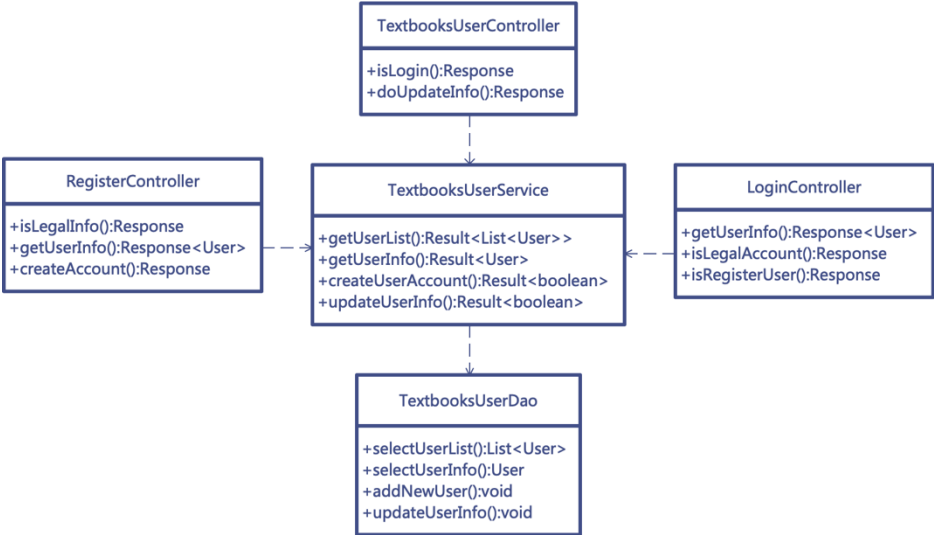


图 4.18 用户服务类图

(3) 用户时序

用户要在进入教材征订系统前注册一个账号并登录。按照提示将个人的用户名，密码与验证码，手机号等填入到正确的位置，随后单击注册按钮，系统会通过 API 网关将请求传送到 RegisterController 类的 isLegal-Info()方法来对其填写信息时的规范性进行检测，以及对其身份和是否被注册过进行检测。如果不能达到规范则返回错误信息，如果达到规范则根据参数使用 createAccout()指定提交至 Service 层的 TextbooksUserService 类中的 createUserAccount()注册用户，最后与数据存储层的 TextbooksUserDao 类交互，在将用户数据存到数据库后返回到客户端，系统对用户进行注册成功的提示并返回到登录界面。可根据上述的流程得到用户注册时序图如图 4.19 所示。

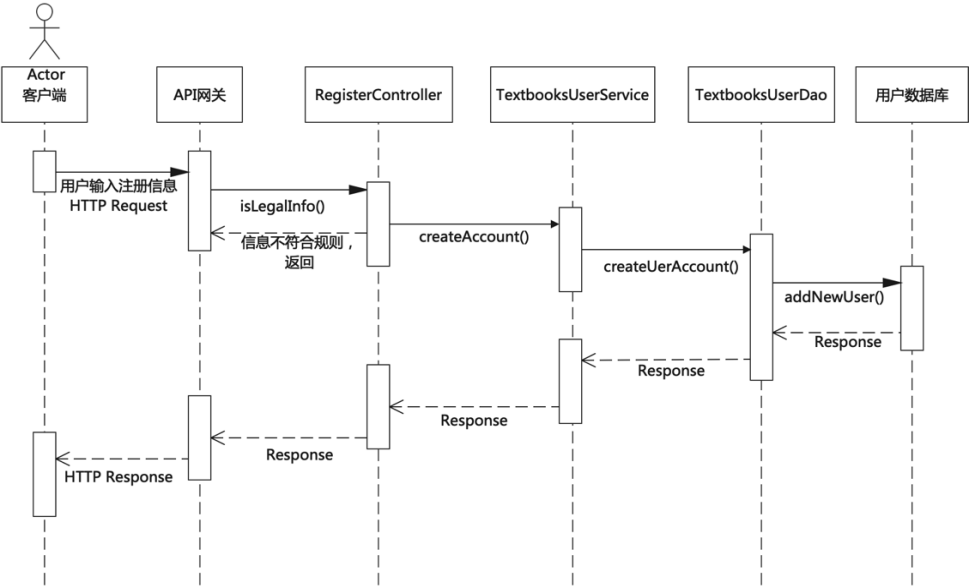


图 4.19 用户注册时序图

登录时，用户在完成表单提交之后，系统通过 LoginController 类获取请求并调用

isLegal-Account()方法核对用户名跟密码,如果判断没有对应的用户名跟密码则直接返回错误。如果判断有这个用户名跟密码,管理系统就会根据这个 username 在数据库中进行查询,若存在改字段则调用 getUserInfo()在数据库中搜寻 userId 字段信息并与用户输入数据做比对,如果一致则返回请求成功消息。此时,用户登录成功并跳转到首页;若信息合演不通过则提示用户重新输入。根据上面的流程能够得到用户登录时序图如图 4.20 所示。

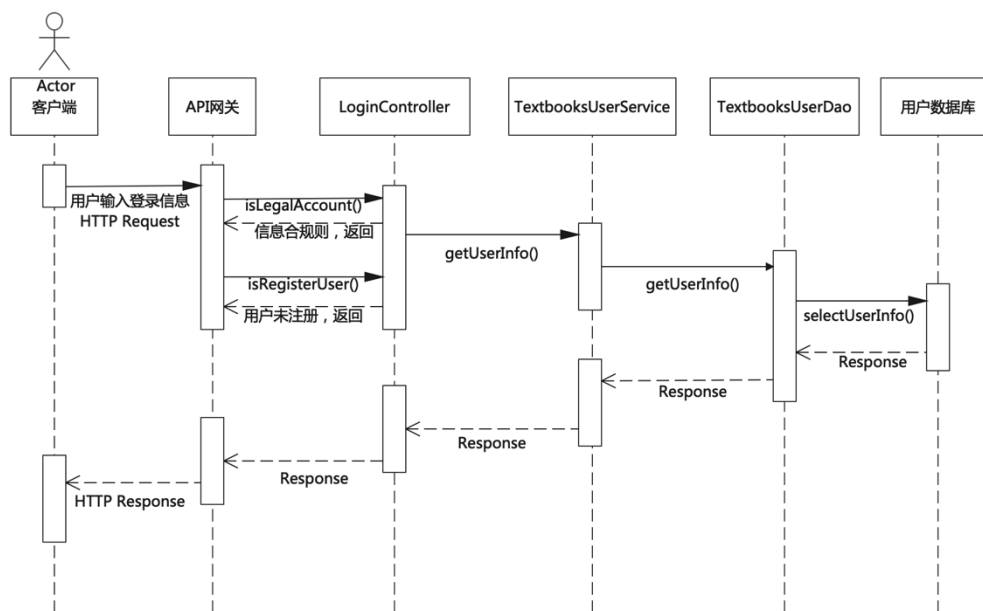


图 4.20 用户登录时序图

用户登录后能够在我的账户界面对个人资料进行修改,用户功能在整个系统贯穿。首先要看是不是登录成功,若根据 TextbooksUserController 中的 isLogin()的判断发现没有成功登录就转换到登录界面告知用户再次登录。用户登录成功之后,单击我的账户中的修改个人信息按钮并将要修改的新信息输入到正确的输入框,此时 TextbooksUserController 类的 doUpdateInfo()方法通调用了 Textbooks-UserService 类中的 updateUserInfo()方法来验证修改的信息是否合规,若符合用 TextbooksUserDao 类就将原本用户的旧数据替换为更新后的数据,如果不符合规定的话就提示用户修改信息错误并抛出异常,至此用户信息的修改流程结束。根据上面的流程可得用户修改个人信息的时序图如图 4.21 所示。

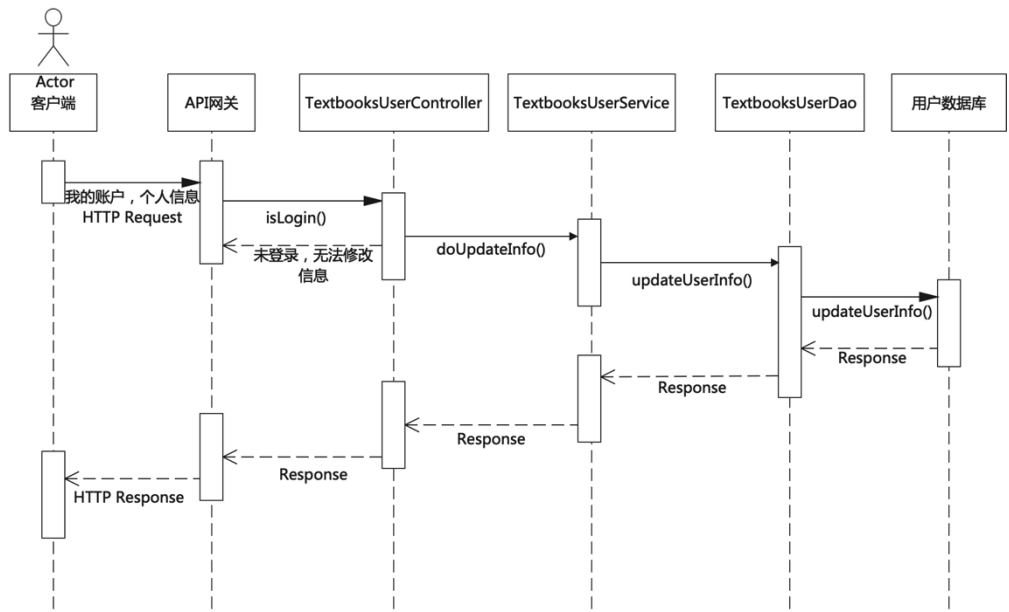


图 4.21 用户修改个人信息时序图

4.5.2 教材服务详细设计与实现

教材征订系统中的教材服务主要有教材搜索、教材信息查看和教材信息更新等功能。

(1) 教材流程设计

以查询教材为例，可以实现多条件多种组合加模糊/精确查询，查询到用户所需要的教材后，将在主程序界面显示查询结果。倘若未找到，则列表显示为空。其流程图如图 4.22 所示。

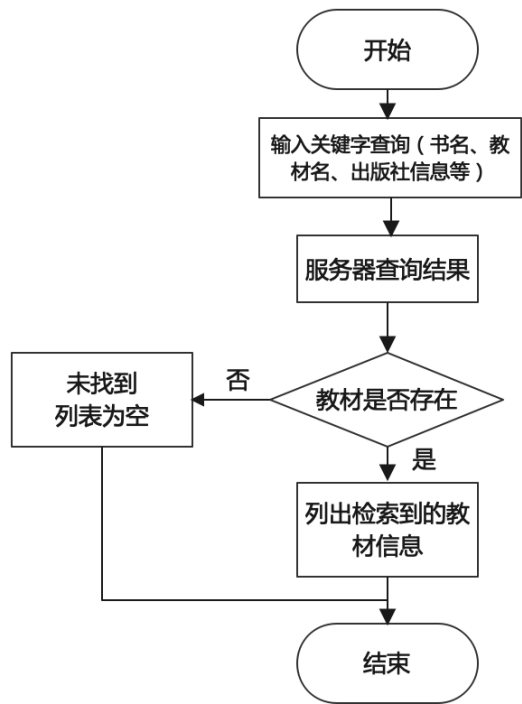


图 4.22 教材查询流程图

教材库存管理模块可以对教材进行入库管理，也可以查询教材的库存情况、出库情况。

教材的入库出库可以通过学校教材科完成,也可以通过经过授权的出版社来完成,两者的操作保证了教材库可以及时得到补充和更新,同时也可以减少教材科相关人员的工作负担。教材科其流程图如图 4.23 所示。

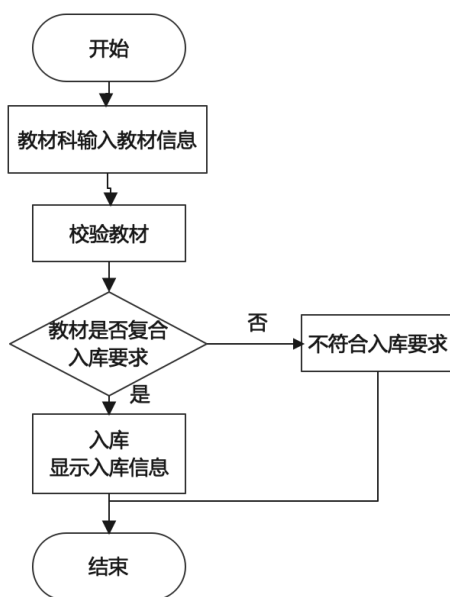


图 4.23 教材科入库流程图

出版社若要进行教材入库需要登录后请求入库授权,若授权通过才能继续教材入库,若授权不通过则需要重新申请授权。通过授权后输入教材信息进行入库操作,若库中存在同名教材,则可通过查看详情来确定是否为相同教材。其流程图如图 4.24 所示。

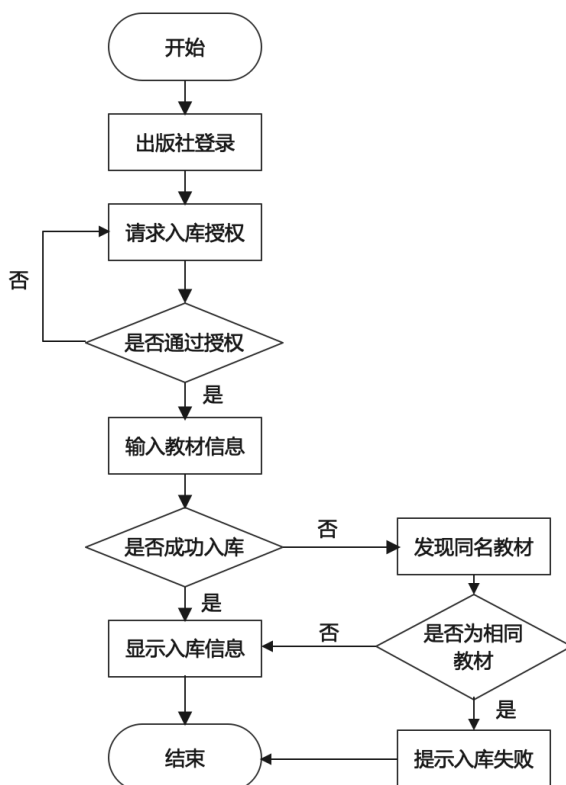


图 4.24 出版社教材入库流程图

(2) 教材类设计

教材服务的类图由 TextbooksController、TextbooksService、TextbooksSearchService 与 TextbooksDao 组成,其中接口层的 TextbooksController 能接收所有关于教材接口调用的请求,booksList()来对教材进行渲染,先查找缓存看里面有没有,如果不能查到就对服务层代码进行调用,查到之后来用 SpringWebContext 进行渲染。TextbooksService 与 TextbooksSearchService 用来与 TextbooksDao 交互查询数据信息。updateBooksInfo()用来更新教材信息包括库存信息等。教材服务类图如图 4.25 所示。

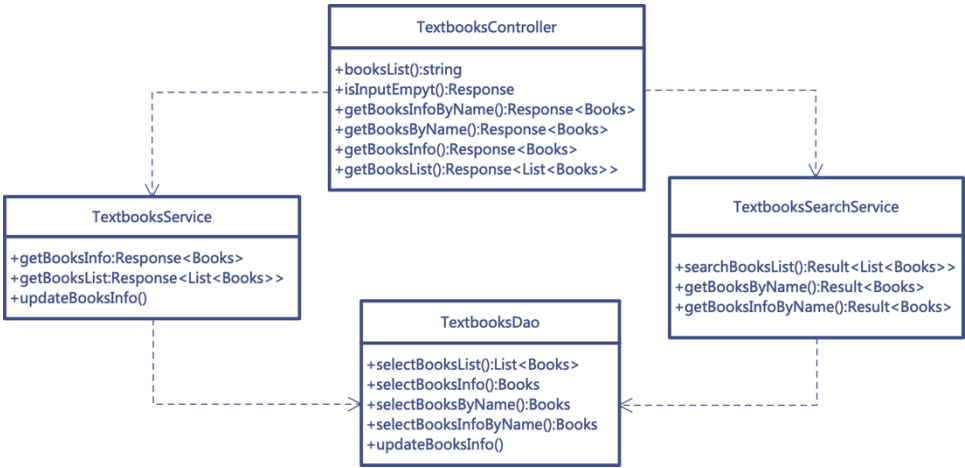


图 4. 25 教材服务类图

(3) 教材时序

本教材征订系统中的教材是根据关键词进行查询搜索的。本系统把一些常用的关键词存到 Redis 高速缓存中去,当用户输入与教材相关的关键字到搜索框之后,首先要用 isInputEmpty()来看用户输入的关键字是不是空的,如果不是空的就用 TextbooksController 中的 goodsList()对 Redis 缓存服务进行调用看有没有存在于高速缓存之中,如果高速缓存之中不存在;再调用 goodsList()方法对缓存服务进行调用来对教材进行查询,若查询不是空的结果就对 CacheService 进行调用来返回结果。如果查询到是一个空的结果则系统对 TextbooksSearchService 中的 search-BooksList()方法进行调用以列表的形式来查询出数据库里搜索的结果,列表里各个元素全都是 Books 对象,调用 TextbooksController 里的 booksList()渲染至客户端界面。由此可得商品查询的时序图如图 4.26 所示。

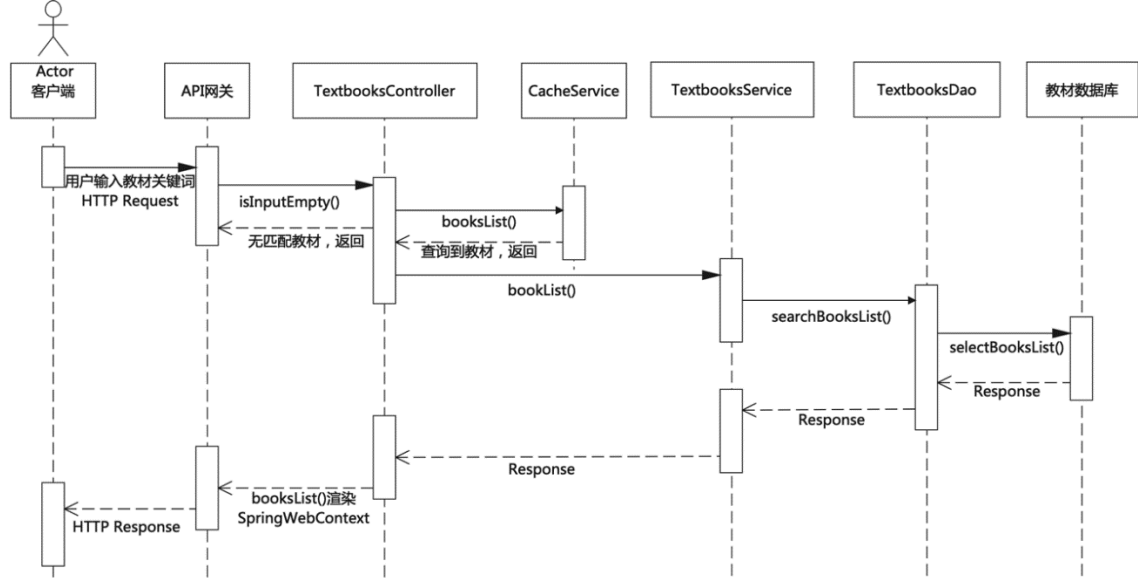


图 4.26 教材查询时序图

4.5.3 书架服务详细设计与实现

书架功能模块用于用户在书架中添加、修改、删除自己所需教材，并可将需要的教材一并结算。

(1) 书架流程设计

用户进入教材征订系统，浏览或查询教材，遇到需要的教材时可添加至我的书架，之后用户可继续浏览，加入我的书架的教材已经添加到 Redis 缓存中，当用户浏览结束后可进入我的书架界面，可以查看到自己书架中的教材，系统会显示教材信息、单价、数量和最终总价。其流程图如图 4.27 所示。

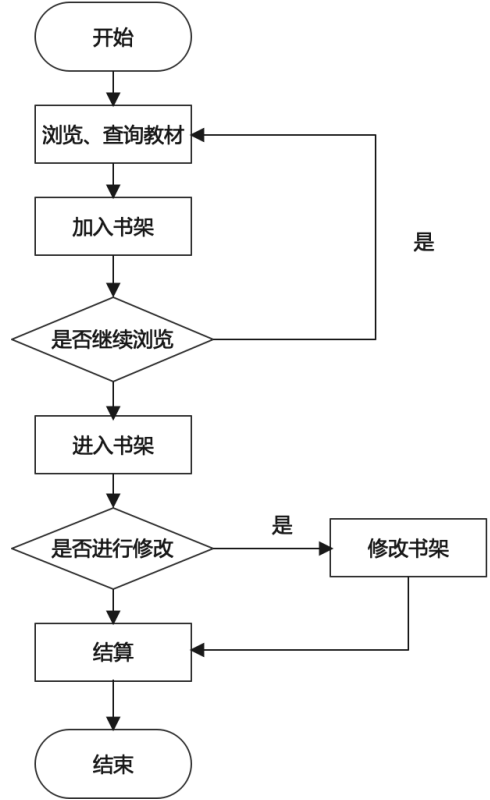


图 4.27 书架添加教材流程图

(2) 书架类设计

书架服务的类图由 BookshelfController、BookshelfService 与 BookshelfDao 组成，BookshelfController 用于对查看、修改书架中信息等请求进行控制。业务层的 BookshelfService 属于实现类，用来实现新增、查看、删除与更新书架中信息功能的编写，TextbooksUserDao 则与数据库交互数据。书架服务的类图如图 4.28 所示。

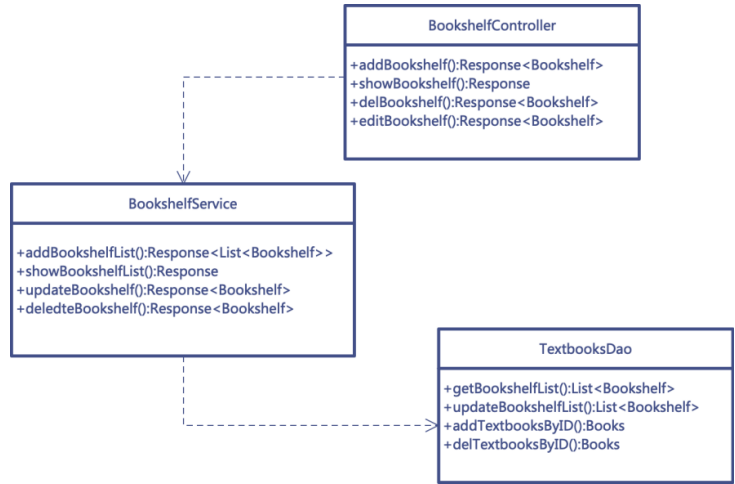


图 4.28 书架服务类图

(3) 书架时序

以添加教材至书架为例，客户端页面发起请求，并将教材信息传到 BookshelfController 层中，调用 addBookshelf()进行添加处理，BookshelfService 中 addBookshelfList()具体进行添

加，在 TextbooksDao 中通过 addTextbooksID 方法从数据库添加教材信息到书架中，最后返回数据。由此可得书架添加教材时序图如图 4.29 所示。

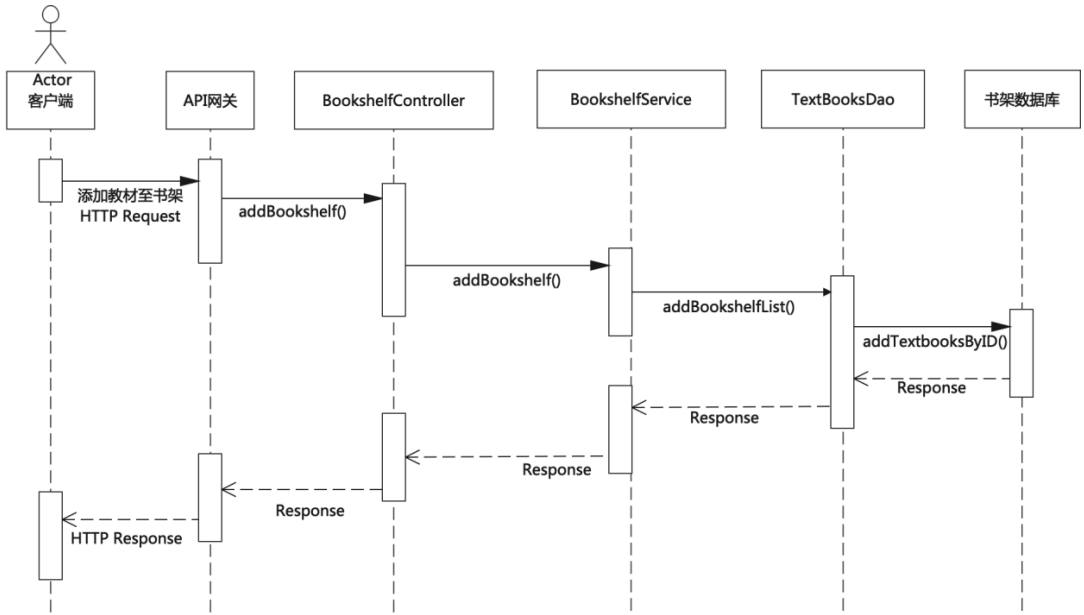


图 4.29 书架添加教材时序图

4.5.4 订单服务详细设计与实现

订单服务各功能与教材服务、书架服务交互，系统生成订单，用户确认后可在教材征订中的查看订单。订单服务功能主要包括订单的创建、查看、取消、删除和审核等。

(1) 订单流程设计

用户登录之后，首先进行查询，符合条件则可提交相关订单信息。而后系统进行库存查询，满足则生成订单，否则给用户二次选择，登记相关信息。教材被用户提交后，教材科将在订单审核模块将审核用户所提交的订单，决定是否予以审核通过，当审核通过后最终生成订单用户。创建流程图如图 4.30 所示。

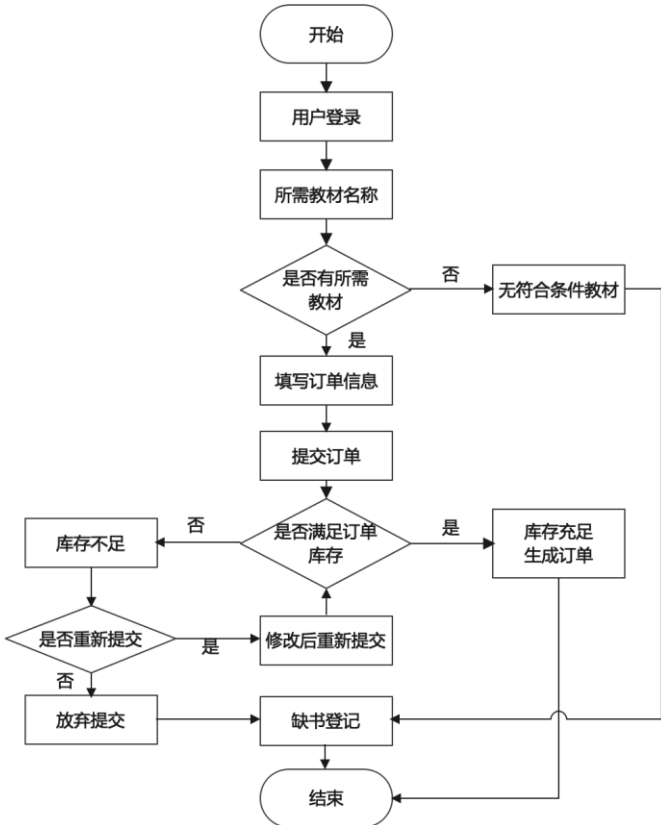


图 4.30 创建订单操作流程

(2) 订单类设计

客户端全部与订单有关的 API 请求被服务端通过 TextbooksOrderController 接收，其中用户查询订单的请求被 getOrderInfo()接收，用户对订单取消的请求被 cancelOrder()接收。订单服务在服务层里与 TextbooksOrderService 跟 CacheService 有关系。而与订单类数据库交互的是 TextbooksOrderDao，其中将系统生成的订单放到数据库中是通过 addOrder()方法实现的；对订单状态进行更新是通过 updateOrder()方法来完成的，订单服务类图如图 4.31 所示。

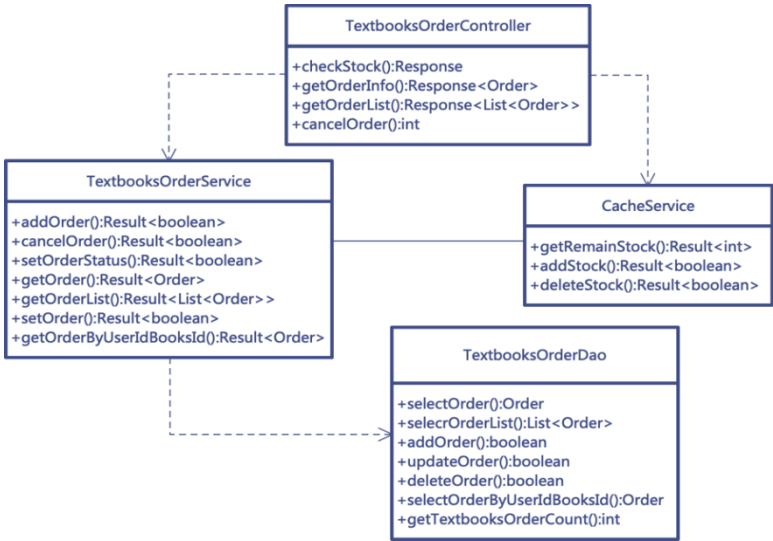


图 4.31 订单服务类图

（3）订单时序

在创建教材订单之初，系统会进行初始化预热这一操作，在 Redis 中缓存教材的全部信息，每次对库存进行查询或者对其增加与减少都要跟 Redis 进行数据交互。系统对 CacheService 中的 getRemainStock()进行调用来对 Redis 里该教材的库存进行检查，如果剩下的库存不够则创建订单失败，请求被丢弃并向用户返回库存不足订单创建失败的提示；若库存充足则返回 TRUE，且调用 reduceStock()将库存减一，接着跳转至下单界面，系统后台通过 addOrder()生成订单，最后在数据库存入新创建的教材订单数据后返回客户端，提示用户教材订单创建成功。由以上可得教材订单创建时序图如图 4.32 所示。

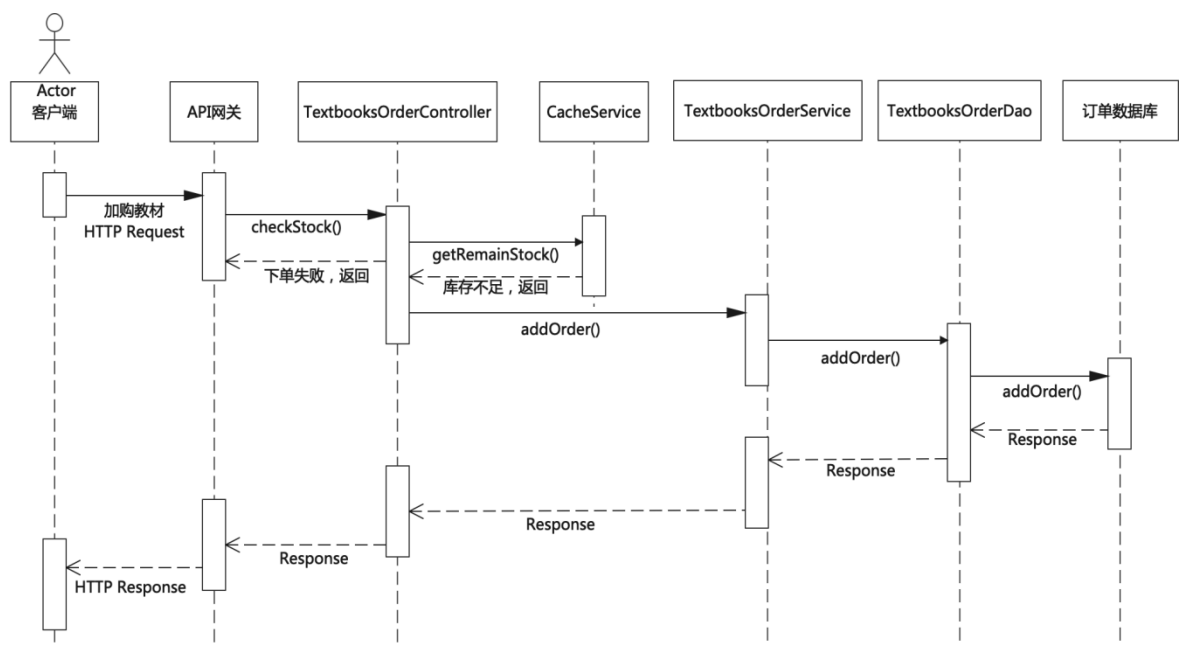


图 4.32 订单创建时序图

4.5.5 评价服务详细设计与实现

评价功能模块用于用户可对我的订单中的教材进行评价。

（1）评价类设计

书架服务的类图由 CommentController、CommentService、CommentDao 组成，CommentController 用于接收查看评价信息等请求。评价服务涉及 CommentService，而跟评价类数据库交互则由 CommentDao 来进行，其中将新增的提交评价放到数据库中则由 addCommen()方法来实现。评价服务的类图如图 4.33 所示。

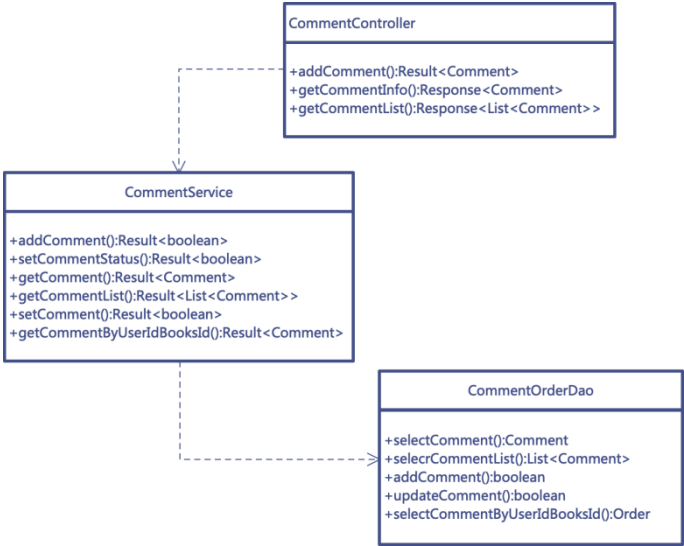


图 4.33 评价服务类图

(2) 评价时序

以提交新增评价为例，用户在客户端页面发起请求，并将教材信息传到 CommentController 层中，调用 addComment() 进行新增评价处理，CommentService 中 addComment() 具体进行添加，系统后台通过 addComment() 生成新增评价，之后将新提交的评价数据存放在数据库后再返回到客户端，将用户评价成功的提示展现给用户。由此可得提交评价时序图如图 4.34 所示。

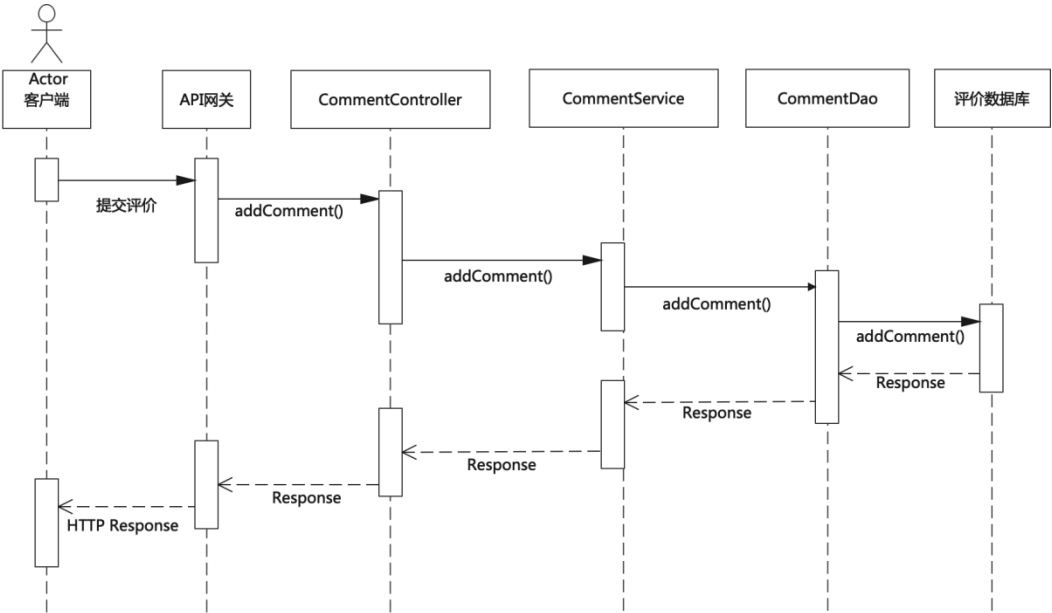


图 4.34 评价提交时序图

4.5.6 通知服务详细设计与实现

在通知服务中所有用户均可下载文件，上传文件因限定上传公告的权限，需要验证其身

份是否符合权限，根据角色权限上传文件。其流程图如图 4.35 所示。

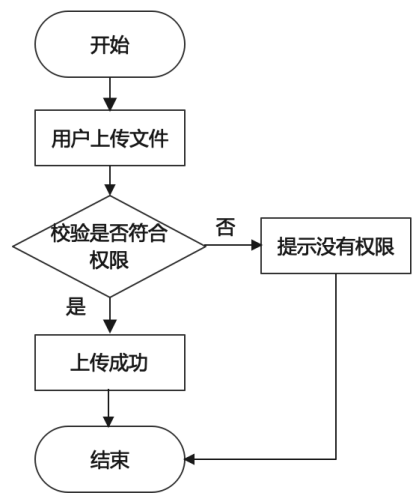


图 4.35 文件上传操作流程

4.6 数据库设计

数据库设计是根据业务的需要在特定的环境下制作出效果最好的数据库关系表，以满足系存储需求和鲁棒性的过程^[67]。本节将按照第三章的需求分析来详细的设计规划系统所需数据库表，不仅满足了系统当下的需求，而且能够应对之后扩展可能带来的问题。

本文采用 MySQL8.0，建立名为 Textbook_Management 的数据库。数据库的结构模型如图 4.36。

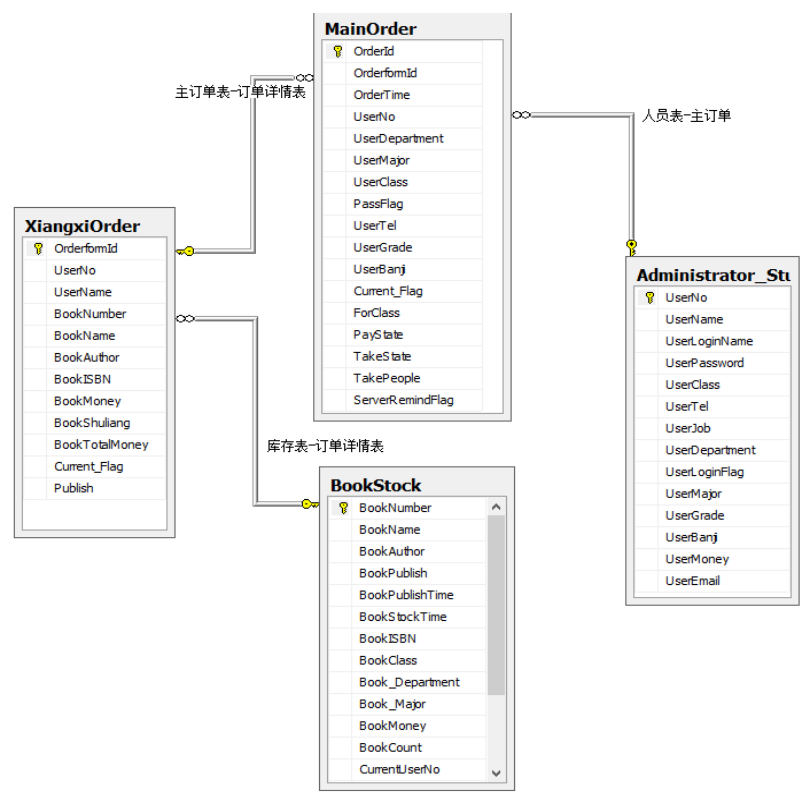


图 4.36 教材征订系统内部数据库模型图

本文所设计的主要数据库表详细名称及其功能如表 4.1 所示。

表 4.1 数据库表及功能

表名	功能
Administrator	管理员表
Administrator_Students_Teachers	用户表
BookStock	教材库存表
BookError	订单错误表
MainOrder	主订单表
XiangxiOrder	订单详情表
QueShuLog	缺书登记表
PublicDownload	公告及文件下载表
School_Department	系别专业表
UploadFiles	教材目录上传表

(1) 管理员表

管理员表用来保存管理员信息。为了系统安全,单独设置了管理员表,其中 UserNo 为自增长字段,主键,用来保存管理员编号,标识唯一性。其具体定义及内容如表 4.2 所示。

表 4.2 管理员表

字 段	类 型	字段长度	是否允许为空	是否主键或外键	说 明
UserNo	int	自增长	不允许	主键	管理员编号
UserName	nvarchar	50	不允许	否	管理员名字
UserLoginName	nvarchar	20	不允许	否	登陆昵称
UserPassword	nvarchar	20	不允许	否	密码
UserClass	nvarchar	50	不允许	否	用户类型
UserTel	nvarchar	15	不允许	否	电话
UserJob	nvarchar	20	不允许	否	工作职务
UserDepartment	nvarchar	20	不允许	否	所在系部
UserLoginFlag	nvarchar	10	不允许	否	登陆标识符
UserMajor	nvarchar	50	允许	否	所在专业
UserGrade	nvarchar	20	允许	否	所在年级
UserBanji	nvarchar	20	允许	否	所在班级
UserEmail	nvarchar	50	不允许	否	电子邮箱

(2) 用户表

用来保存用户信息的表,在该表中存放了用户的详细信息,包括姓名、所在系部、年级、电话等,这些关键的字段用来在用户订购教材时使用。其定义与表 4.2 相同(略)。

(3) 教材库存表

用来保存教材的库存信息,该表存放了教材的详细信息,包括教材的基本信息,入库的时间、入库人、库存数量、默认订购数等一系列关键字段,其中 BookNumber 为自增长字段。

相关具体定义如表 4.3 所示。

表 4.3 教材库存表

字 段	类 型	字段长度	是否允许为空	是否主键或外键	说 明
BookNumber	int	自增长	不允许	主键	教材编号
BookName	nvarchar	100	不允许	否	教材名字
BookAuthor	nvarchar	80	不允许	否	作者
BookPublish	nvarchar	60	不允许	否	出版社
BookPublishTime	datetime		不允许	是	出版时间
BookStockTime	datetime		不允许	否	入库时间
BookISBN	nvarchar	20	不允许	否	ISBN 编号
BookClass	nvarchar	20	不允许	否	书本类别
Book_Department	nvarchar	30	不允许	否	适用系别
Book_Major	nvarchar	50	不允许	否	适用专业
BookMoney	float		不允许	否	定价
CurrentUserNo	int		不允许	否	订单标识符
BookCount	int		不允许	否	库存数量

(4) 订单错误表

订单错误表用来临时存储用户订单提交过程中产生的错误信息，一般是库存不足的书本信息会暂时存储在这里，然后展示给用户。

表 4.4 订单错误表

字 段	类 型	字段长度	是否允许为空	主键或外键	说 明
BookNumber	int		不允许	主键	错误书本编号
BookName	nvarchar	100	不允许	否	书名
BookISBN	nvarchar	20	不允许	否	ISBN 编号
BookAuthor	nvarchar	80	不允许	否	作者
BookPublish	nvarchar	60	不允许	否	出版社
BookPublishTime	datetime		不允许	否	出版时间
BookMoney	float		不允许	否	定价
BookCount	int		不允许	否	库存
Quantity	Int		不允许	否	订购数

(5) 主订单表

主订单表用来存放订单生成时间, 订单序列号, 以便与详细订单表里面的订单进行组合。
具体设计如表 4.5 所示:

表 4.5 主订单表

字 段	类 型	字段长度	是否允许为空	主键或外键	说 明
OrderId	nvarchar	50	不允许	否	主订单编号
OrderformId	int		不允许	外	详细订单编号
OrderTime	datetime		不允许	否	下单时间
UserId	int		不允许	否	订购者编号
UserDepartment	nvarchar	50	允许	否	订购人系别
UserMajor	nvarchar	50	允许	否	订购人专业
UserClass	nvarchar	20	允许	否	用户类型
PassFlag	nvarchar	20	不允许	否	审核通过标识符
UserTel	nvarchar	20	不允许	否	用户电话
UserGrade	nvarchar	20	不允许	否	用户年级
UserBanji	nvarchar	20	不允许	否	用户班级
ForClass	nvarchar	20	允许	否	教材科批量订购
PayState	nvarchar	10	不允许	否	支付标识符
TakeState	nvarchar	10	不允许	否	领书标识符
TakePeople	nvarchar	20	不允许	否	领书人
ServerRemindFlag	int		允许	否	本地服务查询

(6) 详细订单表

详细订单表用来存放所订购书本的详细信息以及订购人编号和姓名, 如表 4.6 所示。

4.6 详细订单表

字 段	类 型	字段长度	是否允许为空	主键或外键	说 明
OrderformId	int		不允许	主键	详细订单编号
UserNo	int		不允许	否	订购人编号
UserName	nvarchar	20	不允许	否	订购人姓名
BookNumber	nvarchar	20	不允许	否	书本编号
BookName	nvarchar	100	不允许	否	书名
BookAuthor	nvarchar	80	不允许	否	作者
BookISBN	nvarchar	20	不允许	否	ISBN 编号
BookMoney	float		不允许	否	定价
BookShuliang	int		不允许	否	订购数量

(7) 缺书登记表

缺书登记表用来存储用户提交的缺书单, 用户提交到这里以后, 教材科管理员可以在这里给用户反馈所提交的教材是否已经入库。具体设计如表 4.7 所示:

表 4.7 缺书登记表

字 段	类 型	字段长度	是否允许为空	主键/外键	说 明
QueShuID	nvarchar	20	不允许	主键	缺书登记编号
BookName	nvarchar	100	不允许	否	书名
BookISBN	nvarchar	20	不允许	否	ISBN 编号
BookAuthor	nvarchar	80	不允许	否	作者
BookPublish	nvarchar	60	不允许	否	出版社
BookPublishTime	datetime		不允许	否	出版时间
BookClass	nvarchar	10	不允许	否	书本类别
RemindMe	nvarchar	10	不允许	否	用户提醒标志符
UserNo	int		不允许	否	登记人编号
UserName	nvarchar	20	不允许	否	登记人姓名
UserDepartment	nvarchar	20	不允许	否	登记人所在系别
UserClass	nvarchar	10	不允许	否	登记人类别
UserTel	nvarchar	20	不允许	否	登记人电话
DengjiTime	datetime		不允许	否	登记时间
ShortMessage	nvarchar	MAX	不允许	否	短消息 (教材科)

(8) 公告及文件下载表

教务处可以在这个表里发布公告和附件, 用户则可以查询和下载。设计如表 4.8 所示:

表 4.8 公告及文件下载表

字 段	类 型	字段长度	是否允许为空	主键或外键	说 明
DocId	nvarchar	20	不允许	主键	公告编号
UserName	nvarchar	20	不允许	否	发布人姓名
ploadTime	datetime		不允许	否	发布时间
GonggaoName	nvarchar	20	不允许	否	公告题目
GonggaoContext	nvarchar	MAX	不允许	否	公告内容
FileName	nvarchar	50	允许	否	附件名
Files	image		允许	否	附件

(9) 教材目录上传表

用来存储教材目录，具体设计如表 4.9 所示：

表 4.9 教材目录上传表

字 段	类 型	字段长度	是否允许为空	主键或外键	说 明
DocId	nvarchar	20	不允许	主键	上传编号
UserId	int		不允许	否	上传者编号
UserName	nvarchar	20	不允许	否	上传者姓名
userDepartment	nvarchar	20	不允许	否	上传者系别
FileName	nvarchar	20	不允许	否	文件名
File_Deparment	nvarchar	20	不允许	否	适用系别
File_Major	nvarchar	50	不允许	否	适用专业
File_Grade	nvarchar	20	不允许	否	适用年级
UploadTime	datetime		不允许	否	发布时间
Files	image		不允许	否	文件
PassFlag	int		不允许	否	审核通过标识符
AttachedFileName	nvarchar	50	允许	否	附件名

4.7 本章小结

本章节主要对教材征订系统的架构进行设计。首先是基于微服务的教材征订系统总体设计；再进一步根据系统的业务功能划分微服务，最终获得六个业务微服务，并对每个微服务用功能结构图加以说明；接着使用原理图叙述了组成教材征订系统服务端各个组件的设计与实现；然后使用类图、时序图、流程图与接口描述的方式详细叙述了系统中各个服务的设计与实现；数据库表则是展示主要字段进行说明论述；从而实现了需求分析中所需的全部功能。

第五章 教材征订系统的测试与效果展示

系统测试分为系统功能测试和性能测试这两个主要部分。功能测试主要包括用户注册登录、教材查询订购、教材评价等项目；性能测试主要包括服务器压力测试和客户端性能测试。由于本文是基于微服务架构开发，同时也对微服务组件进行了测试。

5.1 系统核心功能测试

系统采用了如下的测试环境配置：

1.硬件环境

服务器硬件环境：新华三 R4900G3 服务器主机，2 个 6234CPU，64GB 内存，3.84T SSD 硬盘。

测试客户机硬件环境：联想 510pro，i5-11400，16G 内存，1T 硬盘。

1. 测试客户机软件环境

界面测试工具：Chrome 浏览器

性能测试工具：Apache Jmeter

5.1.1 用户模块测试

（1）注册登录测试

通过测试用户登录模块，来对注册登录功能进行重点测试。当教师（学生）启动系统之后，将进入欢迎和登陆界面。在登陆界面，可以选择学生、教师、系部、出版社等五种用户类型。学生和老师注册时点击登陆界面上的注册按钮进行注册。如果是老师注册则需要管理员审核之后，账号才能激活。表 5.1 是对用户注册登录进行的测试用例，来看看使用者能不能成功登录。

表 5.1 用户注册登录的测试用例

测试用例	用户注册登录
测试功能	不同角色用户注册登录成功进入系统
测试步骤	1. 新用户点击注册，填写相关注册资料； 2. 输入账号密码登录系统。
测试预期	系统验证用户身份，审核通过可登录；登录成功。
测试结果	与预期结果相同
结论	测试通过

用户注册界面效果展示如图 5.1 所示。登录界面如图 5.2 所示，不同用户在登录时可选择与自己账户角色相匹配的选项。

注册 1、填写信息 → 2、管理员审核 → 3、注册成功

用户名:

请输入用户名

*

密码:

请输入密码

*

确认密码:

请输入确认密码

*

邮箱:

请输入邮箱

*

昵称:

请输入昵称

*

真实姓名:

请输入您的真实姓名

*

联系电话:

请输入您的联系电话

*

我已阅读并接受《用户协议》
请阅读协议

提交

图 5.1 用户注册界面展示

教师用户

请输入用户名

请输入密码

请输入验证码

登录

注册

系部用户

✓ 教师用户

学生用户

管理员用户

出版社用户

请输入验证码

5348

登录

注册

图 5.2 用户登录界面展示

(2) 用户信息管理测试

当系统中各个角色的不同用户成功注册登录后，表 5.2 为用户信息管理的测试用例，验证全部用户能否正常使用用户信息管理功能并成功完成个人信息的更新。

测试用例	用户信息管理
测试功能	系统用户完成用户信息管理、修改信息等功能
测试步骤	1. 用户进入用户信息管理功能入口； 2. 用户选择编辑个人信息（跟人信息、头像）； 3. 用户输入并确认更新信息；
测试预期	系统对用户个人信息功能列表进行显示；用户信息显示成可修改状态；系统提示信息更新成功。
测试结果	与预期结果相同
结论	测试通过

用户进入成功登录后进入系统主页效果展示如图 5.3 所示，当选择我的账户时可进入用户信息管理功能。



图 5.3 教材征订系统主页及用户模块展示

5.1.2 教材查询订购模块测试

在本系统中，教材的查询和订购是系统需要实现的主要功能，系统中各个角色的不同用户几乎都需要对教材进行查询。表 5.3 为教材订购查询的测试用例，验证全部用户能否正常使用用查询功能并成功加购所需教材。

表 5.3 教材查询订购的测试用例

测试用例	教材查询订购
测试功能	系统用户完成教材查询、教材订购等功能
测试步骤	1. 用户进入教材征订当中的在线选书入口； 2. 用户输入需要的教材名称，或按类别进行查询； 3. 用户对教材进行选择并且加购； 4. 用户确认提交订单。
测试预期	系统显示教材信息检索列表；系统显示有查询结果的教材信息列表；系统满足库存则可以加购；系统显示提交订单成功。
测试结论	与预期结果相同 测试通过

用户进行在线选书界面展示如图 5.4 所示。用户进入教材征订功能，即可对已知教材进行有针对性的选书查询或根据教材的专业类别进行粗略的查询，之后便可按需求加购，继而生成订单，完成查询订购。完成订购后可返回教材征订功能查看订单。查看订单结果展示如图 5.5 所示。



图 5.4 教材查询订购界面展示



图 5.5 订单查询界面展示

5.1.3 教材评价模块测试

教材评价功能主要针对的是学生和教师这两类用户。教师和学生在规定时间内分别通过自己角色登录系统后对本学期所涉及课程进行教材评价，教师为本学期所授课程教材，学生为本学期所学教材。系统管理人员如教材科、教务处的工作人员可以基于教师与学生的教材评价结果，更有针对性的管理教材目录。表 5.4 为教材评价的测试用例，验证全部用户能否正常使用查询功能并成功加购所需教材。

表 5.4 教材评价的测试用例

测试用例	教材评价
测试功能	系统用户完成教材评价功能
测试步骤	1. 教师和学生用户进入教材评价入口； 2. 教师和学生用户输入需要评价的教材名称； 3. 教师和学生用户选择未评价进行教材评价； 4. 教师和学生用户提交评价。
测试预期	系统显示教材信息列表；系统显示有查询结果的教材信息列表；系统显示被选中教材的评价列表；系统显示提交评价成功。
测试结果	与预期结果相同
结论	测试通过

教师和学生用户进行教材评价展示如图 5.6 所示，还没有评价的教材显示未评价，评价过的教材显示已评价。管理员权限的用户则可以看到教材评价结果，教材评价结果汇总展示如图 5.7 所示。

输入书名、书号或作者 搜索 扫码				
序号	书号	书名	版别	评价
1	9787531024064	中国玉器全集	河北美术出版社	未评价
2	9787122025203	中国蝴蝶观赏手册	化学工业出版社	未评价
3	9787122225429	商业插画设计	化学工业出版社	未评价
4	7539808292	中国飞天艺术	安徽美术出版社	已评价

图 5.6 教材评价界面展示

查看评价

valuation list

您当前所在的位置：教材评价 > 查看评价

请输入书名、书号

导出

序号	书号	书名	版本	定价	订数	教A	教B	教C	学A	学B	学C	操作
1	7539808292	中国飞天艺术	安徽美术出版社	254.4	17	100%			88.7%	11.3%		详情
2	7800475905	清代宫廷版画	紫禁城出版社	21.4	17				78.3%	21.7%		详情
3	9787539815787	国画入门训练	安徽美术出版社	12.0	45	28%		72%	75.8%		24.2%	详情
4	97787563502356	信息科学原理	邮电大学出版社	28.0	61	50%	50%		67.2%		32.8%	详情
5	9787563559985	现代通信技术	邮电大学出版社	49.0	30	60%	40%			61%	39%	详情

图 5.7 教材评价汇总界面展示

对于系统的用户注册登录、教材订购查询、教材评价等功能，根据以上设计的测试用例步骤，完成了系统功能测试。所有的测试用例均通过测试，主要功能都能够较好的实现，表明系统按照需求完成了所有功能。

5.2 服务组件测试

对微服务基础组件进行测试，主要是对网关、注册中心和配置中心这些微服务基础组件进行测试，观察其能否正常运行。当这些组件运行无误后，才算完成系统的微服务环境构建。

5.2.1 服务注册中心测试

作为服务发现组件的服务注册中心，服务注册中心掌管着各启动后的微服务的注册与认证权限并将已经被注册的服务访问地址进行保存，测试用例如表 5.5 所示。

表 5.5 服务注册中心测试

测试用例	Eureka 测试
测试目的	考察系统的注册中心的运作情况，是否正常
测试前提	启动 Eureka 微服务
测试过程	1. 对 Configserve 和 EurekaServer 进行启动操作； 2. 对每个微服务工程依次启动； 3. 将地址输入到浏览器中
测试预期	在地址被输入浏览器后，可以通过 Eureka 管理界面对所有注册微服务的实例名进行查看。
测试结果	测试成功

5.2.2 网关服务测试

网关服务需要完成对前后端进行解耦的操作，前端通过先对网关进行访问然后再连接到目标服务，测试用例如表 5.6 所示。

表 5.6 服务网关测试

测试用例	Zuul 网关服务测试
测试目的	能否利用 Zuul 网关服务访问对用户服务
测试前提	对 Eureka 和微服务进行启动
测试过程	1. 对 Configserve 和 EurekaServer 进行启动； 2. 先对用户服务进行启动，然后启动网关服务； 3. 对指定端口进行访问
测试预期	网关服务把请求发送到用户服务，初始化成功。
测试结果	测试通过

5.2.3 配置中心服务测试

服务配置中心服务对所有服务的配置文件进行管理，在对微服务进行启动操作的时候都要从配置中心服务里面对配置文件进行读取，完成配置并启动。测试用例如表 5.7 所示。

表 5.7 配置中心服务测试

测试用例	配置中心服务测试
测试目的	对是不是可以在配置中心里对配置文件进行获取并成功启动进行测试
测试前提	对 Eureka 和微服务进行启动操作
测试过程	1. 启动 Configserve 和 EurekaServer； 2. 对用户服务进行启动操作。
测试预期	配置中心成功对配置文件进行读取，启动成功。
测试结果	测试通过

5.3 系统性能测试与结果

5.3.1 服务端测试

系统服务端性能测试主要在服务器端处来观察系统能承受的高并发访问量，通过响应时间的平均值来对系统的高并发业务处理能力进行衡量。现在市面上有着各种各样的测试系统压力的软件，本文在测试中选择了 Apache Jmeter 测试工具来实时模拟用户的高并发，对系统的教材服务、评价服务以及订单服务进行考虑。特别是评价服务，大量师生用户在规定短时间内统一完成本学期的教材评价，由于评价服务有着较大的并发量，所以主要测试这部分的接口压力。同时，为了验证本文设计的教材征订系统面对并发业务的优势，作者使用 JAVA 语言将基于微服务架构的教材征订系统部署 4 台 PC 主机上并与基于 MVC 模式的单体架构教材征订系统对比。具体测试如表 5.8 所示。

表 5.8 微服务架构系统压力测试表

主要接口	10s 内线程并发数	平均响应时间 (ms)	最小响应时间(ms)	最大响应时间(ms)	请求错误率 (%)
Order	500	26	10	66	0.00
	1000	89	11	322	0.00
	2000	2574	13	4862	4.15
	3000	4788	12	8562	14.1
Book	500	35	12	74	0.00
	1000	104	14	378	0.00
	2000	3122	14	6634	5.5
	3000	5477	13	9577	17.3
Comment	500	44	14	95	0.00
	1000	127	16	452	0.00
	2000	3946	16	7321	15.5
	3000	5888	18	12588	29.0

表 5.9 MVC 模式单体架构系统压力测试表

主要接口	10s 内线程并发数	平均响应时间 (ms)	最小响应时间(ms)	最大响应时间(ms)	请求错误率 (%)
Order	500	31	8	86	0.00
	1000	97	10	411	0.00
	2000	2884	10	5342	4.7
	3000	5120	12	9652	17.7
Book	500	40	11	97	0.00
	1000	132	13	464	0.00
	2000	3570	13	8584	11.5
	3000	5961	15	11287	27.7
Comment	500	49	13	109	0.00
	1000	159	13	498	0.01
	2000	4297	16	9192	18.9
	3000	6414	18	14225	32.1

通过观察两种架构的压力测试表，测试订单服务、评价服务和教材服务的主要接口所承受的压力，按 500、1000、2000 及 3000 进行递增 10 秒区间对线程组参数进行设置。从学校实际出发，全校师生人数相对有限，与在校师生的实际人数结合起来对模拟用户并发访问量进行设置。从表格的结果来看，系统的各业务模块的重要接口在 10 秒内并发量不超过 2000 的时候有着较短的平均响应时间，与当下的项目需求相符。当有着超过 3000 的并发访问量时，接口有着比较长的响应时间。

通过对比实验发现，基于微服务架构的教材征订系统在高并发情况下的三个模块的最大响应时间比 MVC 架构和传统单体架构的响应时间要大，主要原因是 API 网关和架构控制面转发业务的处理时间，对单个请求的处理时间稍长，但从长期来看，微服务架构在处理多个请求时，有最小的最大响应时间，这表示所有服务获得满足所需最短时间。同时观察发现，随着最大响应时间的增加，请求错误率也随之增加，因此，采用微服务架构通过降低最大响应时间，从而降低因为延时而出现的错误请求，这也是系统产生错误请求的主要原因。最后，微服务架构在同等并发任务数量下相较于 MVC 模式的传统单体架构在不同模块下的平均响应时间最低。因此基于微服务架构的教材征订系统在应对高并发业务时更具有优势。

5.3.2 客户端测试

为了验证用户在使用了基于微服务的教材征订系统后获得了良好的使用体验，本文针对最容产生高并发情况的评论模块，在模拟 1200 个并发任务和同等网络条件下，通过帧数分析软件查看由教师与学生组成的 30 个用户集的客户端在分布式系统和传统单体系统下的单个

页面跳转时间。页面平均跳转时间如表 5.10 所示：

表 5.10 用户页面跳转时间测量表

用户	微服务架构平均页面跳转时间（ms）	传统单体架构平均页面跳转时间（ms）
用户 1	2509	2983
用户 2	2864	2851
用户 3	2615	2936
用户 4	2525	2979
用户 5	2254	2568
用户 6	2871	2568
用户 7	2654	2987
用户 8	2508	3068
用户 9	2214	2574
用户 10	2541	2574
用户 11	2344	2651
用户 12	2541	2458
用户 13	2874	2689
用户 14	3202	3102
用户 15	2578	2683
用户 16	2468	2787
用户 17	2445	2661
用户 18	2698	2769
用户 19	2758	2824
用户 20	2445	2607
用户 21	2365	2902
用户 22	2459	3008
用户 23	2335	2569
用户 24	2415	2758
用户 25	2374	2566
用户 26	2581	2687
用户 27	2451	2884
用户 28	2156	2486
用户 29	2109	2665
用户 30	2645	2884

为了验证调研数据的普适性，微服务架构的客户端是否比传统架构在高并发条件下的页面跳转时间有显著差异。本文采用 Python 分析数据，进行 χ_2 检验。作出假设：

H0：假设微服务架构系统客户端的页面平均跳转时间相较于传统单体架构系统客户端的页面平均跳转时间没有显著差异。

H1：假设微服务架构系统客户端处的页面平均跳转时间相较于传统单体架构系统客户端

的页面平均跳转时间存在显著差异。

根据表 5.12 的结果，采用微服务架构系统的客户端平均跳转时间为 (2526.6 ± 229.4684) ms，而采用传统单体架构系统的客户端为 (2757.6 ± 179.9252) ms，同时观察到 $P=0.000074381 < 0.05$ ，因此接拒绝原假设 H_0 ，接受假设 H_1 ；即微服务架构系统客户端处的页面平均跳转时间相较于传统单体架构系统客户端的页面平均跳转时间存在显著差异；且微服务架构系统客户端的页面跳转时间低于微服务系统客户端。因此，采用微服务的方式优化传统单体系统同时能在用户侧给用户带来更好的使用体验。

表 5.12 χ_2 检验结果表

	人数	页面平均跳转时间 (ms)
微服务架构	30	2526.6 ± 229.4684
传统单体架构	30	2757.6 ± 179.9252
T 值		-4.266
P 值		$0.000074381 < 0.05$

5.4 本章小结

本章主要对教材征订系统的各功能进行测试并展示效果，对系统的用户注册登录模块、教材查询订购模块、评价模块进行了详细的功能测试。设计用例对微服务基础组件进行测试看它能否正常运行，测试通过表明成功的对微服务环境进行搭建。最后从压力测试方面对系统的性能进行测试，也完成了最初的预期。

第六章 总结与展望

本章对整篇文章的主要内容进行总结，主要有研究工作，同时对未来进一步的优化做出展望。

6.1 总结

教材建设是改革教育思想、教学内容、教学方法不可忽视的环节，也是实现教育现代化的重要环节。当今世界日新月异，进步如歌，尤其体现在软件行业，快速迭代的业务需求以及无法预知的变化都强迫着人们不断地对软件适应外部变化的能力进行提升，开发者都为了研发更强适应性、更强扩展性架构而肝脑涂地，这些努力增强了系统的生命力以及灵活性，微服务架构便是这些理论和工程实践的集大成者。从刚开始对微服务进行关注，到现在国内许多互联网公司已经接受并使用微服务架构，并且许多互联网公司都为微服务设计了相关组件。作为一名软件工程专业的硕士研究生，本人在读研期间有幸接触到微服务并对其有着浓厚的兴趣，因此，在设计和实现项目的时候，选择了微服务架构而不是单体架构，结合自身所在学校教务处的本职工作，通过微服务架构模式来对教材征订系统进行设计并开发。

主要工作归纳如下：

(1) 研究了微服务的理论以及方法，通过学习相关的知识，论述了单体架构到的微服务架构的转变过程与技术细节，对如何设计微服务有了更加深入的了解，例如按照业务逻辑划分、领域驱动设计等。

(2) 分析教材征订系统的需求以及对其架构进行设计，以了解的软件工程知识为基础，详细的对系统需求进行建模，第三章就是以此为基础的。之后与需求模型相结合，对业务功能进行设计，参照业务功能，本文应用系统被划分为六个业务服务以及三个基础组件服务，并对服务架构进行设计。再之后使用 Spring Cloud 来对微服务环境进行构建，构建例如 Eureka 服务注册中心、Config 配置中心、Zuul 服务网关等等微服务基础组件。然后使用类图、时序图、流程图与接口描述的方式详细对各个服务的设计与实现进行了叙述。

(3) 对教材征订系统的三个主要模块进行了详细的功能测试。接着通过设计用例来看看微服务基础组件可不可以正常的运行，保证了业务功能服务的可用性和可靠性。

总而言之，与传统单体架构设计的教材征订系统相比较，以微服务架构为基础设计的教材征订系统有很多的优点：

(1) 可扩展性: 当要新增一个相对来说独立的功能时, 可以将其实现成一个新的服务, 再集成到系统里面, 通过调整少量的接口的逻辑, 就能够很容易扩展系统功能。

(2) 负载均衡: 启动很多个实例在某个服务器上, 并且将这些实例均匀的在不同服务器上部署, 当某个服务实例出故障后立刻对新的服务实例进行调用, 以此来确保系统的正常运行。

(3) 实现职能划分: 划分服务就相当于划分职能。开发团队能够独立地对各自负责的微服务进行开发、测试、部署和维护, 给团队的每一员划分了职责, 有利于团队进行开发工作。

同样, 有利有弊, 微服务架构也带给系统许多新的复杂的东西, 例如:

(1) 工作量增加: 微服务架构与单体架构相比较, 开发人员要考虑更多的因素进行开发, 比如服务识别、服务测试和服务部署等, 使得工作量大大增加。

(2) 服务管理: 大大增加了管理难度, 微服务系统与只管理一个项目的单体系统相比, 要在分布式环境下对多个微服务进行管理。

(3) 服务可靠性: 带来了很大的挑战给系统容错性, 服务间由于越来越小的颗粒度划分, 不可避免的存在级联依赖, 当一个服务出现故障崩掉的时候, 就有可能带来多米诺效应对整个系统带来影响。

微服务架构在带来巨大优势的同时也不可避免的带来一些缺点, 要根据实际情况来判断要不要采用微服务去开发软件, 要从实际出发, 对利弊进行权衡, 作出一个最合适的选择。本文之所以采用微服务架构去开发教材征订系统, 主要是方便以后对征订系统进行扩展, 而单体架构没有这样的功能。

6.2 展望

本文的微服务架构模式基于 Spring Boot 和 Spring Cloud 实现, 虽然对主要功能进行了设计并实现, 但整个系统的架构设计和实现还有很多改进空间。

现阶段的系统, 完成了教材征订的基本功能, 后续计划是关联学籍系统、教务系统。目前系统只实现了六个业务微服务, 离模拟分布式系统复杂性实验的要求还有一定距离, 因此对一些由分布式所带来的微服务热点问题的考虑还略有欠缺, 希望未来可以对整体系统架构有进一步的完善。

性能方面, 作为整个系统的入口网关 Zuul 网关, 对系统全部的访问需求有着不可或缺的承担作用, 若 Zuul 网关出现问题, 系统则无法正常工作。之后研究方向将选择部署网关集群来改善 Zuul 网关性能或是选用其他网关。

参考文献

- [1] XU Jingwei. Research on the Operational Management Mechanism of Newly Established Undergraduate Colleges[J]. HEILONGJIANG EDUCATION (Higher Education Research & Appraisal), 2017, 31(7): 62-63.
- [2] LUO Nianlong, CHEN Huaichu, ZOU Xiangrong, ZHONG Wenfeng. Resource- based Integrated Graduate and Undergraduate Academic Management System[J]. Modern Educational Technology, 2014, 25(4): 104-111.
- [3] 王璟. 天津电大新华分校网上教材征订系统的设计与实现[D]. 成都:电子科技大学, 2013: 3-4.
- [4] 吴化尧, 邓文俊. 面向微服务软件开发方法研究进展[J]. 计算机研究与发展, 2020, 57(03): 525-541.
- [5] Malavalli D, Sathappan S. Scalable microservice based architecture for enabling DMTF profiles[C]//2015 11th International Conference on Network and Service Management (CNSM). IEEE, 2015: 428-432.
- [6] 崔海涛, 章程, 丁翔, 等. 面向微服务架构的开发组织适应性评估框架[J]. 软件学报, 2021, 32(5): 1256-1283.
- [7] Alshuqayran N, Ali N. Evans R. A systematic mapping study in microservice architecture[C]//2016 IEEE 9th International Conference on Service-Oriented Computing and Applications (SOCA). IEEE, 2016: 44-51.
- [8] Dragoni N, Giallornzo S, Lafuente A, et al. Microservices: Yesterday, today, and tomorrow[J]. Present and Ulterior Software Engineering, 2017, 4: 195-216.
- [9] 辛园园, 钮俊, 谢志军, 等. 微服务体系结构实现框架综述[J]. 计算机工程与应用, 2018, 54(19): 10-17.
- [10] Thönes J. Microservices[J]. IEEE Software, 2015, 32(1): 116.
- [11] 施凌鹏, 朱征, 周俊松, 等. 面向微服务架构的云系统负载均衡机制[J]. 计算机工程, 2021, 47(9): 44-50, 58.
- [12] M. Viggiano, R. Terra, H. Rocha, M. T. Valente, and E. Figueiredo. Microservices in Practice: A Survey Study[C]. 2018.
- [13] Dimensional Research: GLOBAL MICROSERVICE TRENDS-A survey of development professionals[C]. 2018.
- [14] 牛会生. 基于Web的高校教材管理系统的设计与开发[D]. 秦皇岛:燕山大学图书馆, 2017.
- [15] 过晓娇. 基于Java EE和JQuery的高校教材管理系统设计与开发[D]. 广州:中山大学图书馆, 2012.
- [16] 胡廷柱. 基于B/S的高校教材征订管理信息系统研究[J]. 电脑知识与技术, 2022, 18(16): 48-50.
- [17] 朱翔宇, 谢小芳, 潘桃桃, 等. 基于ASP.NET的高校教材管理系统的开发与设计[J]. 电脑知识与技术, 2021, 17(23):72-74.
- [18] 周立. Spring Cloud与Docker微服务架构实战[M]. 电子工业出版社, 2017:30-50.
- [19] Li Jinfeng, Sun Bingcheng. Research on Information Services Integrated Method Based on SOA[J]. Journal of Physics: Conference Series, 2021, 1802(3): 18-19.
- [20] M. Richards. Microservices vs. Service-Oriented Architecture[M]. O'Reilly Media, Inc. 2016:5-8.
- [21] S. Newman. Building Microservices[M]. O'Reilly Media. 2014:12-13.
- [22] L. Chen. Continuous Delivery: Huge Benefits, but Challenges Too[C]. IEEE Softw. vol. 32, no. 2.2015.
- [23] L. Chen. Microservices: Architecting for Continuous Delivery and DevOps[C]. In 2018 IEEE International Conference on Software Architecture (ICSA), Seattle, WA. 2018.
- [24] S.Newman.Building microservices: designing fine-grained systems. First Edition[M]. Beijing Sebastopol, CA: O'Reilly Media. 2015:21-22.
- [25] A. Sill. The Design and Architecture of Microservices[C]. IEEE Cloud Comput, vol. 3, no. 5. 2016.
- [26] Balalaie A, Heydarnoori A, Jamshidi P. Microservices architecture enables devops: Migration to a cloud-native architecture[J]. IEEE Software, 2016, 33(3): 42-52.
- [27] M. Fowler and J. Lewis. Microservices a definition of this new architectural term[M]. O'Reilly Media,

- Inc.2018:35-38.
- [28] Zhang J, Wu X, Zhang Y, et al. Research on SFC Deployment with Variable Granularity based on Microservice Architecture[C]//2021 IEEE 3rd International Conference on Frontiers Technology of Information and Computer (ICFTIC). IEEE, 2021: 478-482.
- [29] P. D. Francesco, I. Malavolta, and P. Lago. Research on Architecting Microservices: Trends, Focus, and Potential for Industrial Adoption[C]. in 2017 IEEE International Conference on Software Architecture (ICSA), Gothenburg, Sweden. 2017.
- [30] N. Dragoni. Microservices: Yesterday, Today, and Tomorrow[M]. in Present and Ulterior Software Engineering. M. Mazzara and B. Meyer, Eds. Cham: Springer International Publishing. 2017:14-17.
- [31] Ciuffoletti A. Automated deployment of a microservice-based monitoring infrastructure[J]. Procedia Computer Science, 2015, 68: 163-172.
- [32] 徐言. 基于微服务架构的Web AR服务平台中关键技术的研究与实现[D]. 北京:北京邮电大学图书馆, 2019.
- [33] 肖仲垚. 微服务通信框架的设计与实现[D]. 北京:北京交通大学图书馆, 2017.
- [34] 陈春霞. 基于容器的微服务架构的浅析[J]. 信息系统工程, 2016, 3: 95-96.
- [35] Newman S. Building microservices: designing fine- grained systems[M]. [S.l.]: OReilly Media, Inc, 2015:27-34.
- [36] Amaral M, Polo J, Carrera D, et al. Performance evaluation of microservices architectures using containers[C]//2015 IEEE 14th International Symposium on Network Computing and Applications. IEEE, 2015: 27-34.
- [37] Marfil D, FBoronat, Gonzalez J , et al. Design and Assessment of a Scalable and Customizable Low-Cost Tiled Display System[J]. IEEE Latin America Transactions, 2021, 19(5):708-716.
- [38] Puspitasari N, Budiman E, Sulaiman Y N, Firdaus M B. Microservice API Implementation For E-Government Service Interoperability[J]. Journal of Physics: Conference Series, 2021, 1807(1): 36-38.
- [39] 方永敢. 微服务架构的研究及小区生活服务平台的实现[D]. 成都:电子科技大学图书馆. 2020.
- [40] Sahin Omur, Akay Bahriye. A Discrete Dynamic Artificial Bee Colony with Hyper-Scout for RESTful web service API test suite generation[J]. Applied Soft Computing Journal, 2021, 104.
- [41] Salikhov D, Khanda K, Gusmanov K, et al. Microservice-based iot for smart buildings[J]. arXiv preprint arXiv: 1610.09480, 2016.
- [42] N. M. Coelho, M. C. Cunha, L. G. Ferreira, Security in Microservices Architectures[J]. Procedia Computer Science, 2021, 181: 1225-1236
- [43] 毕小红, 刘渊, 陈飞. 微服务应用平台的网络性能研究与优化[J]. 计算机工程, 2018, 44(5): 53-59.
- [44] Villamizar M, Garces O, Ochoa L, et al. Infrastructure cost comparison of running web applications in the cloud using AWS lambda and monolithic and microservice architectures[C]//2016 16th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid). IEEE, 2016: 179-182.
- [45] Granchelli G, Cardarelli M, Di Francesco P, et al. Towards recovering the software architecture of microservice-based systems[C]//2017 IEEE International Conference on Software Architecture Workshops (ICSAW). IEEE, 2017: 46-53.
- [46] Saxena Jitendra,Goel Aditya. Differentiated services and its importance in fault tolerant WDM optical network[J]. Optical and Quantum Electronics, 2021, 53(3).
- [47] 龙新征, 彭一明, 李若淼. 基于微服务框架的信息服务平台[J]. 东南大学学报 (自然科学版), 2017, 1.
- [48] 张真. 基于微服务架构的项目管理系统设计与实现[D]. 成都:电子科技大学图书馆, 2020.
- [49] 刘耀方. 基于微服务的广电节目推荐平台的设计与实现[D]. 成都:电子科技大学图书馆, 2020.
- [50] Xiangjing Hu, Shugang Liu. Design and Implementation of Student Grade Analysis System Based on Spring Boot Microservice Framework[J]. International Core Journal of Engineering, 2019, 5(10).

- [51] He S, Zhao L, Pan M. The Design of Inland River Ship Microservice Information System Based on Spring Cloud[C]//2018 5th International Conference on Information Science and Control Engineering (ICISCE). IEEE, 2018: 548-551.
- [52] Du Y, Sun B. Remote Monitoring System of Intraocular Pressure Fluctuation Based on SpringCloud[C]//Proceedings of the 2nd International Symposium on Artificial Intelligence for Medicine Sciences. 2021: 268-273.
- [53] 周永圣, 侯峰裕, 孙雯, 等. 基于SpringCloud微服务架构的进销存管理系统的设计与实现[J]. 工业控制计算机, 2018, 31(11): 129-130, 133.
- [54] Balalaie A, Heydarnoori A, Jamshidi P. Microservices architecture enables devops: Migration to a cloud-native architecture[J]. Ieee Software, 2016, 33(3): 42-52.
- [55] 李苏璇. 基于微服务架构的SaaS应用构建方法研究[D]. 广州: 华南理工大学图书馆, 2016.
- [56] 吴雄劲. 微服务框架中的服务注册中心的设计与开发[J]. 工业控制计算机, 2021, 34(8): 130-132.
- [57] 洪华军, 吴建波, 冷文浩. 一种基于微服务架构的业务系统设计与实现[J]. 计算机与数字工程, 2018(1): 149-154.
- [58] Jia Liang. Research on an Optimization Algorithm of Load Balancing based on Ribbon[J]. International Core Journal of Engineering, 2021, 7(8):
- [59] 王佩华. 微服务架构深度解析[M]. 电子工业出版社. 2021: 149-153.
- [60] 崔韵鹏. WEB集群系统负载均衡技术及其算法研究[D]. 大连理工大学, 2017.
- [61] 谭畅, 谭歆, 胡磊, 熊炜. 云中心基于Nginx的动态权重负载均衡算法[J]. 重庆邮电大学学报(自然科学版), 2021, 33(06): 991-998.
- [62] JZeng C, Luo L, Zhang T, et al. Tiara: A scalable and efficient hardware acceleration architecture for stateful layer-4 load balancing[C]//19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22). 2022: 1345-1358.
- [63] Bhattacharya R, Wood T. BLOC: Balancing Load with Overload Control In the Microservices Architecture[C]//2022 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS). IEEE, 2022: 91-100.
- [64] 赵文硕. 关系型与非关系型数据库的应用研究[D]. 北京: 华北电力大学图书馆, 2016.
- [65] Wardiana T, Nurohman F, Samsudin I, et al. Kinerja Database Cache Menggunakan Memcached dan Redis pada Load Balanced Server NGINX dengan Metode RoundRobin. 2021.
- [66] 梅巧玲, 杨立鹏, 樊春美, 等. 基于Redis技术的常用联系人集群架构优化[J]. 铁路计算机应用, 2018, 27(10): 41-45.
- [67] T. Yoon and B.K. Jeong. Service Oriented Architecture (SOA) Implementation: Success Factors and Realized Benefits[J]. International Journal of Information Systems in the Service Sector, 2018.